

Plan du cours

Introduction générale sur les structures de données

Généralités

Objets

Objets typés

Actions élémentaires sur les objets

Les structures de données statiques

Définition

Structures de données homogènes

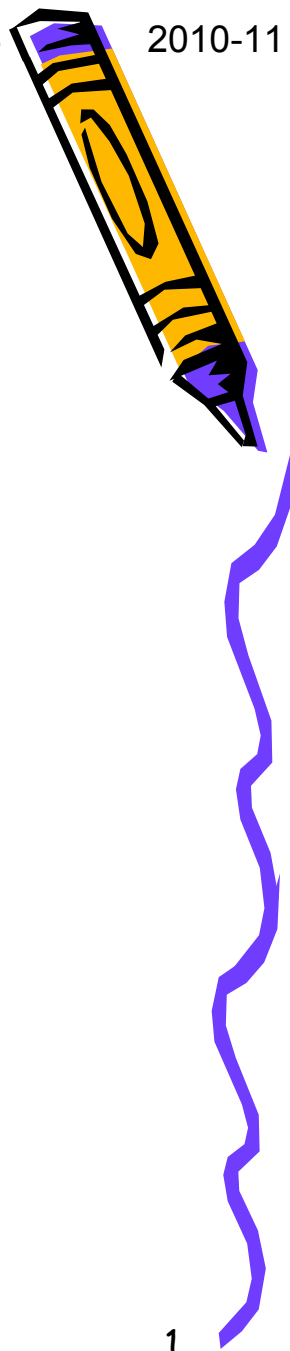
Définitions

Les structures de données dynamiques

Structures dynamiques

Listes / Files / Piles

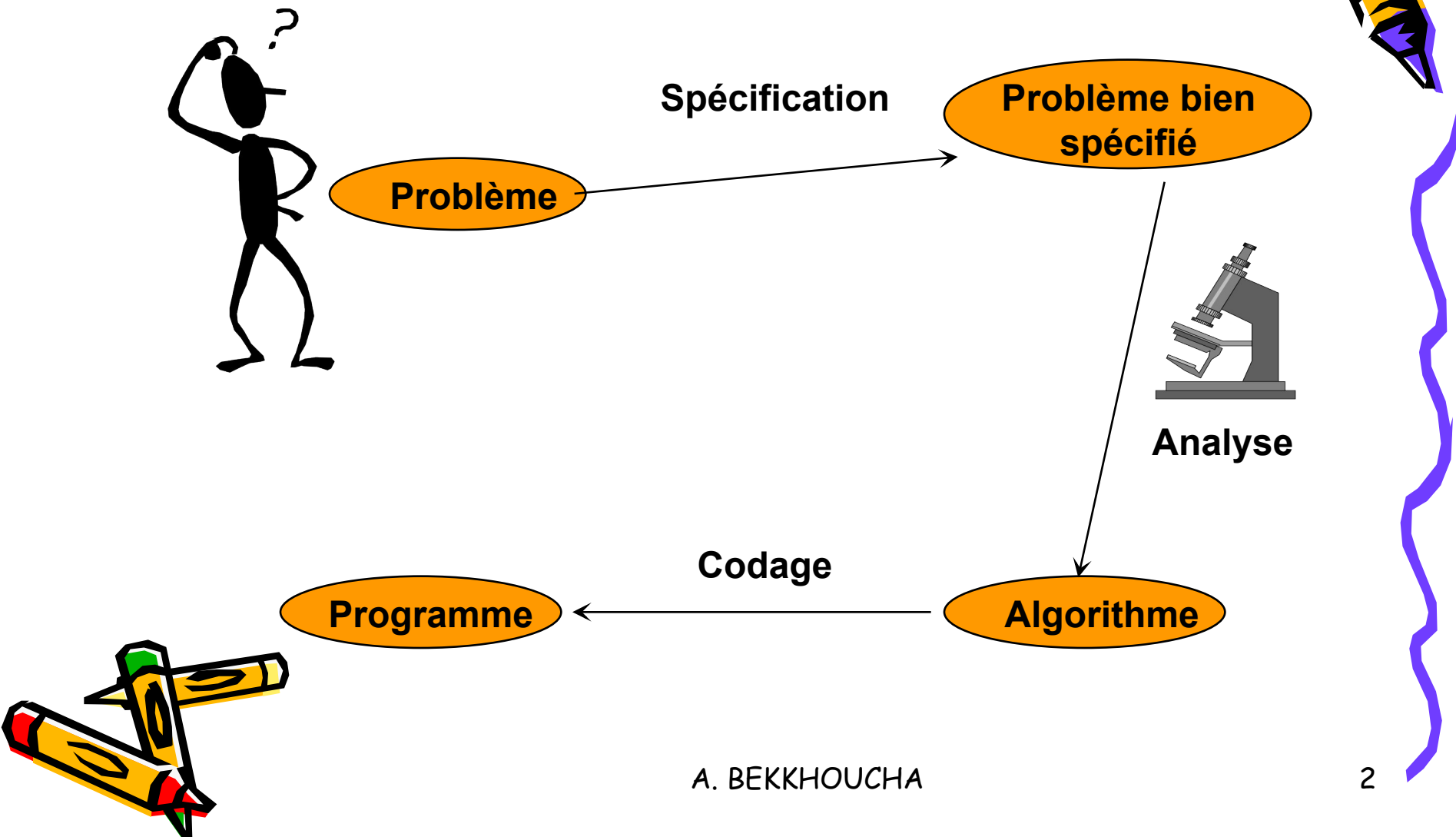
Les arbres



Introduction générale sur les structures de données

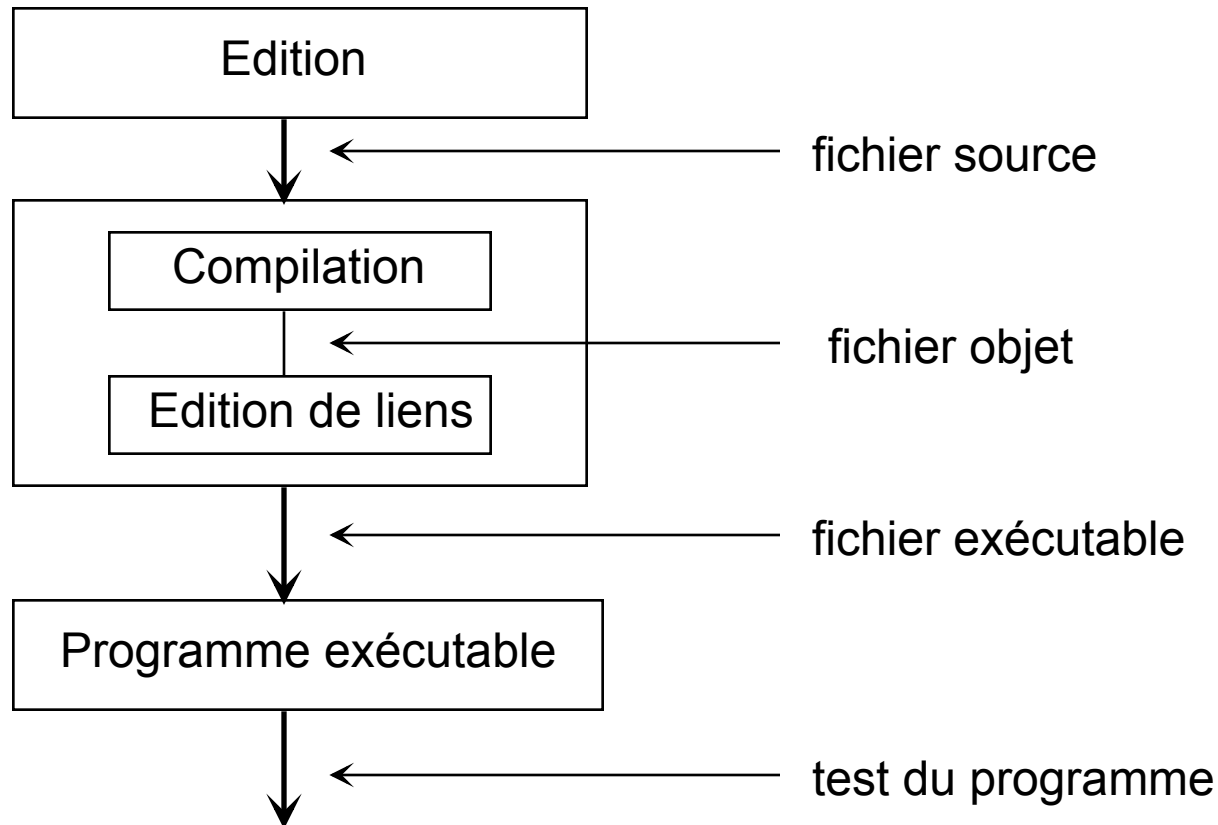
Généralités

Langage C : Algorithme et programme



Généralités

Le codage



Introduction générale sur les structures de données

Généralités

Langage C : Algorithme et programme

Gestion des erreurs sur un programme

Syntaxique

point virgule manquant
parenthèse ouvrante ou fermante manquante
structure de contrôle mal écrite
.....

Sémantique

fonction non déclarée
bibliothèque non trouvée
.....



Introduction générale sur les structures de données

Objets

Langage C : Nom

Identificateur : c'est un nom qui est attribué à une donnée. Ce nom doit répondre à des critères.

Identificateurs correctement écrit:

n
n1
numero_1,
NUMERO_1 // En C il y a une différence entre les Maj. et les Mini.
Nombre_de_caracteres

Identificateur incorrectement écrit:

_nom_var	// pas des "_" en début d'identificateur
1n	// pas de chiffre au début
numero-1	// pas de caractères spéciaux au début
#_de_caracteres	// des identificateurs
Nombre de caracteres	// pas d'espaces



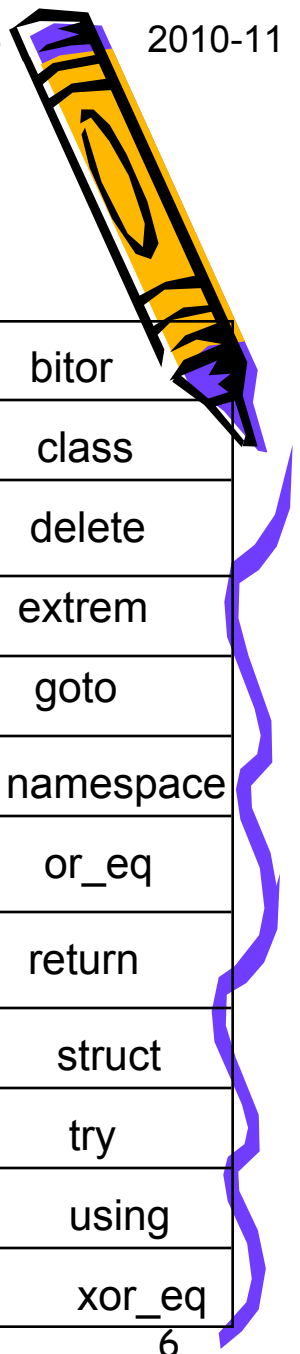
Introduction générale sur les structures de données

Objets

Langage C : Nom

Mots clés en C++ réservés ne peuvent être utilisé par le programmeur

and	and_eq	asm	auto	bitand	bitor
bool	break	case	catch	char	class
compl	const	const_cast	continue	default	delete
do	double	dynamic_cast	else	enum	extrem
explicit	false	float	for	friend	goto
if	inline	int	long	mutable	namespace
new	not	not_eq	operator	or	or_eq
private	protected	public	register	reinterpret_cast	return
short	signed	sizeof	static	static_cast	struct
switch	template	this	throw	true	try
typedef	typename	typeid	unsigned	union	using
virtual	void	volatile	while	xor	xor_eq



Introduction générale sur les structures de données

Objets

Type

Un type de données détermine l'ensemble des valeurs auquel appartient une constante, ou qui peuvent être prises par une variable ou une expression, ou qui peuvent être engendrées par un opérateur ou une fonction.

Un type **simple** contient une **seule** information

Un type **structuré**, relatif à un seul individu, contient un ensemble d'informations réparties en plusieurs champs. Ces champs sont accessibles individuellement ou collectivement. Ils sont identiques pour les champs homogènes et différents pour ceux hétérogènes.



Introduction générale sur les structures de données

Objets

Langage C : Nature

donnée ou *algorithme*

Une *constante* a une valeur compatible avec son type, définie une fois pour toute et inchangeable.

Une *variable* peut prendre n'importe quelle valeur compatible avec son type, cette valeur pouvant changer pendant l'exécution des actions de l'algorithme.

Une *procédure* est un algorithme qui à partir de données d'entrées produit un ou plusieurs données sorties, alors qu'une *fonction* produit une valeur.



Introduction générale sur les structures de données

Objets typés

Types de données simples fondamentaux et prédéfinis

Les constantes:

Constantes non nommées entières

123 // décimale: ne commence jamais par un zéro
0147 // octale commence toujours par un zéro
0x17a // une hexadécimale commence toujours par
0X17A // 0x ou 0X

Remarques:

- 1- une constante négative est définie par l'ajout du signe "-".
exemples : -12, -0x7
- 2- une constante peut être définie de taille long en ajoutant
"l" ou "L" à celle-ci
exemples : 45l, 85L
- 3- une constante peut être non signée.
exemples : 1245U, 789UL (constante de type unsigned long int)

Introduction générale sur les structures de données

Objets typés

Types de données simples fondamentaux et prédéfinis

Les constantes:

Constantes non nommées à virgule flottante

Syntaxes:

1- PartiEntière_Décimales[Exposant]

exemples: $1.23 = 12.3e-1 = 12.3E-1$

2- _Décimales[Exposant]

exemples: $_.123$, $_.147e2$

3- PartieEntière_[Exposant]

exemples: $123_$

4- ChiffresExposant

exemples: $123000e-3 = 123$

Constantes caractères

'a' // le caractère a

'A' // le caractère A

'1' // le caractère 1

Introduction générale sur les structures de données

Objets typés

Types de données simples fondamentaux et prédéfinis

Les constantes:

Caractères spéciaux ou séquences d'échappement

Caractère	code ASCII	Séquence d'échappement
'	39	\'
"	34	\"
\	92	\\
?	63	\?
caractère nul (NULL)	0	\0
Signal sonore (BEL)	7	\a
Retour arrière (BS)	8	\b
Tabulation Horizontale(HT)	9	\t
Saut de ligne (NL/LF)	10	\n
Tabulation verticale (VT)	11	\v
Saut de page (FF)	12	\f
Retour Chariot (CR)	13	\r

Introduction générale sur les structures de données

Objets typés

Types de données simples fondamentaux et prédéfinis

Les constantes:

Caractères chaînes

" une chaîne de caractères entre guillemets"

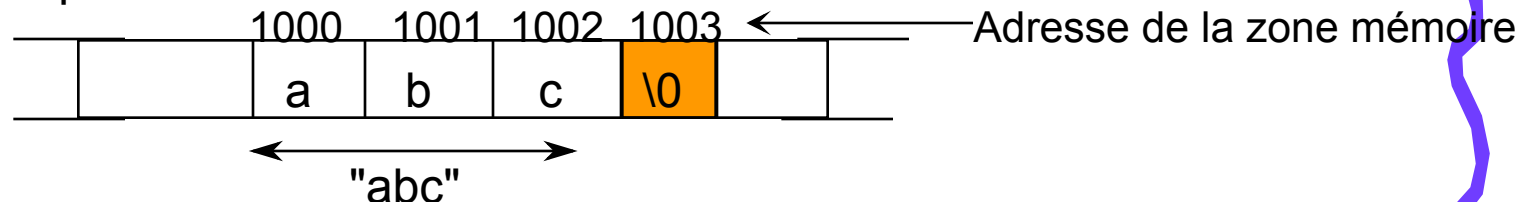
Attention à l'insertion de séquence d'échappement dans les chaînes de caractères

Exemple:

`\7, \07, \007` // sequence representant le caractère BEL

`"\07123"` // le programmeur s'attend a un bip sonore et
// à l'affichage de 123 mais il récupère
// la valeur 923. Le neuf correspond à `\071`

Représentation d'une chaîne en mémoire:



A. BEKKHOUCHA

Objets typés

nis

2010-11

13

Syntaxes:

```
exemple :    #define Pi 3.14159  
              #define Euro 10.75
```



Objets typés

2010-11

nis

ar

ded char

14

A yellow pencil with a purple eraser and a purple band is shown at an angle, drawing a wavy purple line. The pencil has black outlines and a yellow body. The wavy line is a thick, purple stroke that starts from the tip of the pencil and extends downwards. The background is white.

Type caractère **Mot clé** : **char**
Taille d'un entier: **8 bits**

Un caractère peut être:	
- signé signed char	- non signé unsigned char



Structure d'une donnée de type *signed char*



Introduction générale sur les structures de données

Objets typés

Types de données simples fondamentaux et prédéfinis

Types entiers	Mot clé : int
	Taille d'un entier: 16 bits

Des variantes du types entiers existent:

char	: 8 bits
short int	: 16 bits
long int	: 32 bits

Un entier peut être:

- signé

signed int
signed short int
signed long int

- non signé

unsigned int
unsigned short int
unsigned long int

Un entier peut être déclaré simplement par:

short
long

Introduction générale sur les structures de données

Objets typés

Types de données simples fondamentaux et prédéfinis

Type à virgule flottante

structure générale:

représentation interne d'un nombre à virgule flottante



Exemple:

le nombre 12.345 s'écrit en notation normalisée $0.12345 * 10^2$

Pour gérer les nombres à virgule flottante nous disposant, en c++, de trois types : float, double, long double.



Introduction générale sur les structures de données

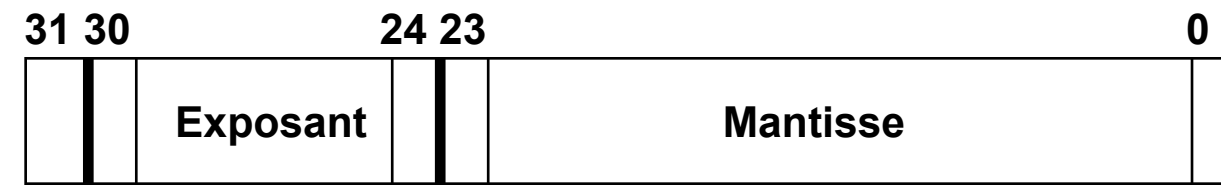
Objets typés

Types de données simples fondamentaux et prédéfinis

Type à virgule flottante: type float

Mot clé : float

Taille d'un réel : 32 bits



- signe : 1 bit
- exposant : 7 bits
- mantisse : 24 bits



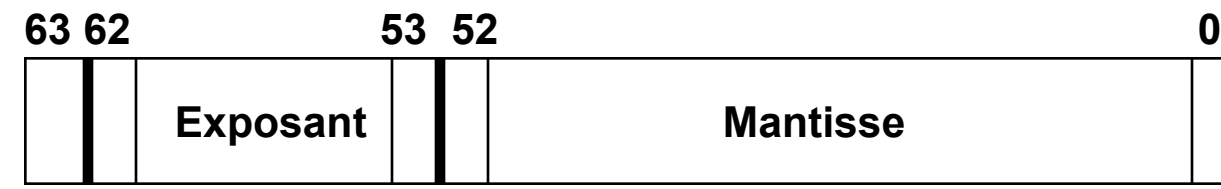
Introduction générale sur les structures de données

Objets typés

Types de données simples fondamentaux et prédéfinis

Type à virgule flottante: type **double** et **long double**

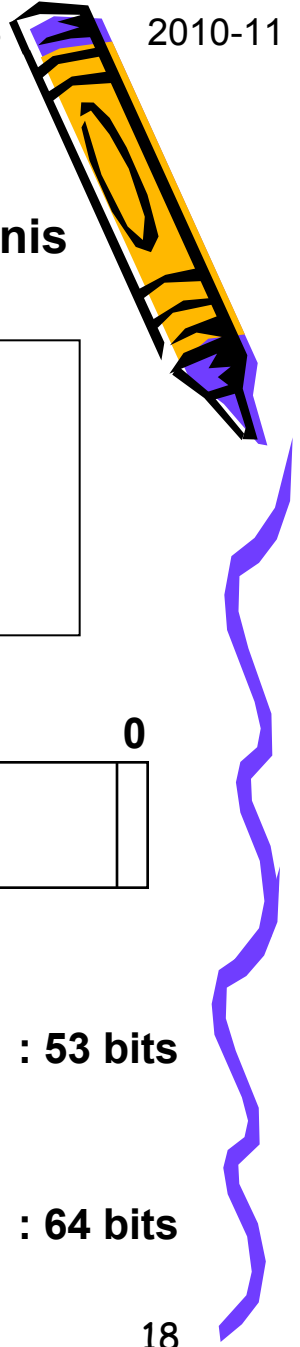
Mot clé : **double**
Taille d'un réel : **64 bits**



signe : 1 bit exposant : 10 bits mantisse : 53 bits

Un **long double** est de taille de 10 octets

signe : 1 bit exposant : 15 bits mantisse : 64 bits



Introduction générale sur les structures de données

Objets typés

Types de données simples fondamentaux et prédéfinis

Types de données fondamentaux du C++

Type	Taille (octet)	Intervalle de valeurs	Précision (chiffres)
char	1	-128 à 127	-
unsigned char	1	0 à 255	-
int	2 ou 4	comme short ou long	-
unsigned (int)	2 ou 4	comme unsigned short ou long	-
short	2	-32768 à +32767	-
unsigned short	2	0 à 65535	-
long	4	-2147483648 à 2147483647	-
unsigned long	4	0 à 4 294 967 295	-
float	4	$-3.4 \cdot 10^{-38}$ à $+3.4 \cdot 10^{38}$	6
double	8	$-1.7 \cdot 10^{-308}$ à $+1.7 \cdot 10^{308}$	15
long double	10	$-1.2 \cdot 10^{-4932}$ à $+1.2 \cdot 10^{4932}$	18

Introduction générale sur les structures de données

Objets typés

Types de données simples fondamentaux et prédéfinis

Les variables :

Syntaxe :

```
type nom_variable1 [, nom_variable2,...];
```

Exemple:

```
char    Un_caractere;  
int     val_ent;  
int     val_ent1,  val_ent2;  
short int petit_entier;  
float   Un_floattant;  
double  UnDouble, DeuxDouble;
```

// Déclarations et initialisations

```
int     val = 10,          autre_val = 20;  
char    a_Minuscule = 'a', A_Majuscule = 'A'
```



Introduction générale sur les structures de données

Objets typés

Types de données simples fondamentaux et prédéfinis

Type énuméré

Le type énuméré est défini par la liste de toutes les valeurs possibles pour les données de ce type. La définition est donnée par

TYPE nom_de_type =(valeur1, valeur2, ; valeurN)

Les éléments de l'énumération sont munis d'un ordre :
valeur1 < valeur2 < < valeur

Syntaxe (langage C):

Enum Nom_enum {val1, val2,....., valN} [var1, var2, ...]

Exemples

enum chiffres	[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
enum voyelles	['a', 'o', 'e', 'i', 'y']

Introduction générale sur les structures de données

Objets typés

Types de données simples fondamentaux et prédéfinis

Type intervalle

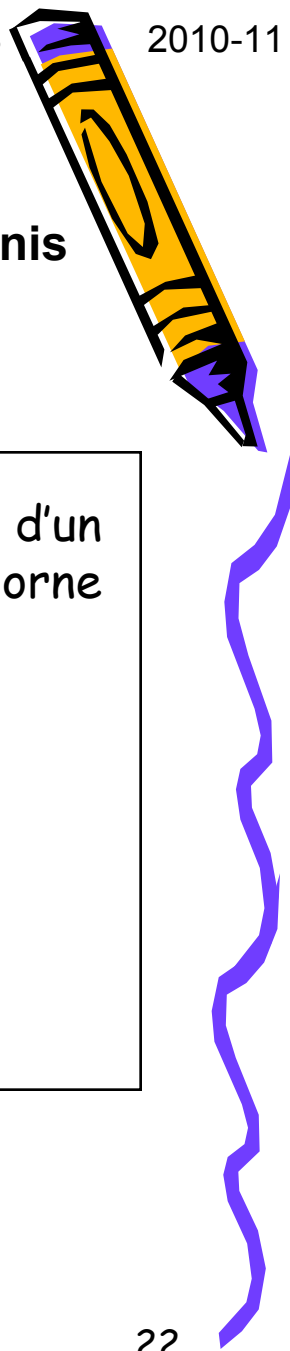
Une valeur peut appartenir à un sous-ensemble réduit d'un ensemble. Ce sous ensemble est délimité par une borne inférieure et une borne supérieure défini comme suit :

TYPE non_intervalle = [inf.. sup]

Exemple

TYPE Annee = [1900..2099]

TYPE Lettre = ['A'..'Z']



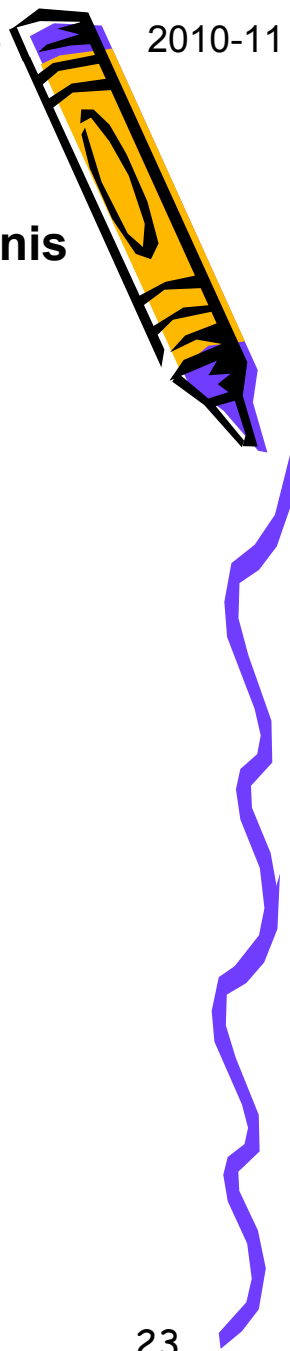
Introduction générale sur les structures de données

Objets typés

Types de données simples fondamentaux et prédéfinis

Type structuré

Cette partie sera abordée ultérieurement



Introduction générale sur les structures de données

Langage C : Structure générale d'un programme en C

/* Déclaration des librairies */

```
# include < stdio.h >  
# include < stlib.h >  
.....
```

// déclaration des librairies standards à l'aide
// des deux chevrons "<" et ">"

```
# include " MyLib1 "  
# include " MyLib2 "  
.....
```

// déclaration des librairies personnelles
// à l'aide des deux guillemets "nom_librairie"

/* déclaration des constantes*/

```
# define    Nom_Const    Val_Const  
# define    Nom_fonct    Corps_fonct
```

/* Définition des types et des structures*/

/* Déclaration des variables globales seulement si nécessaire */

```
Type_prédéfini  nom_variable ;  
Type_défini      nom_var ;
```

A. BEKKHOUCHA

Introduction générale sur les structures de données

Langage C : Structure générale d'un programme en C

/* Déclaration des fonctions */

```
type_fonct  Nom_fonction([ les paramètres])  
{  
  /* corps de la fonction de structure identique a celui  
    du programme principal  
  */  
}
```

/* Le programme principal */

```
type  main( )  
{  
    /* définition de constantes*/  
  
    /* définition de types*/  
  
    /*déclaration des variables locales*/  
  
    /* instructions en langage C*/  
}
```

**Fonction principale
du programme**

A. BEKKHOUCHA

Introduction générale sur les structures de données

Langage C : Structure générale d'un programme en C

Exemple de programme en Langage C

programme: premier_prog.cpp

```
# include < stdio.h>    // définition de la fonction printf() dans la librairie standard

void main()    // void indique que le type de la fonction n'a pas d'importance
{
    /* première instruction en C*/

    printf(" mon premier programme en C");

}
```

Remarque: un programme doit être bien documenté
avec des commentaires.



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Ensemble des opérateurs



Opérateurs d'affectations



Opérateurs arithmétiques



Opérateurs de comparaison



Opérateurs logiques



Opérateurs de bits



Opérateurs de décalage



Incrémentation et décréments



Opérateur d'adressage



Les parenthèses



Conversions de type



Autres opérateurs



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Opérateurs d'affectations

Affectations simples: elles affectent avec l'opérateur = et transmettent la valeur de l'opérateur **droit** à l'opérateur **gauche**.

 Syntaxe :

$$Lvariable = \begin{cases} valeur \\ expression \end{cases}$$

 Exemples :

```
int i, j, a;           // déclaration des variables
i = 5;                 // affectation de la valeur 5 à la variable i
j = 3*2;               // affectation du résultat de l'expression à la variable j
a = i+j;               // affectation du résultat de l'expression à la variable a
i = j = a = 10;        //affectation de la valeur 10 aux variables i, j, et a
```



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Opérateurs d'affectations: combinées

Les affectations combinées associent les opérateurs d'affectation à des opérateurs binaires ou arithmétiques. Ils permettent de raccourcir l'écriture séparée des deux opérations contenues.

 Syntaxe :

Lvariable op =

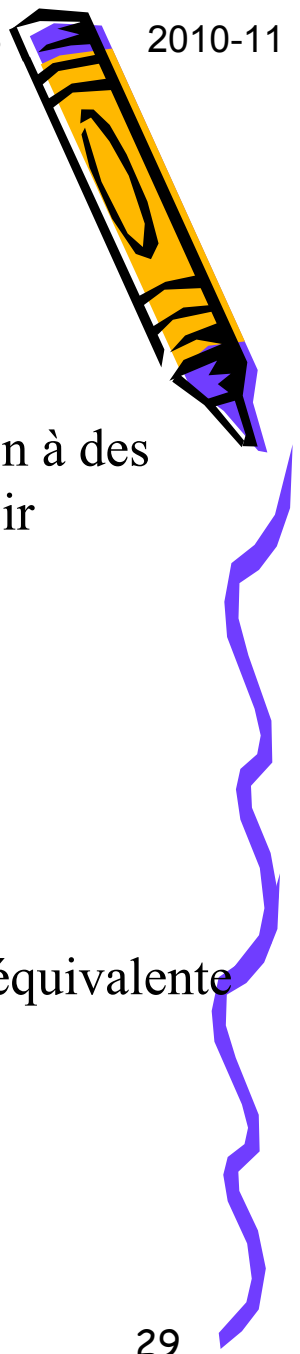
valeur

expression

 Exemples :

Affectation combinée	affectation détaillée équivalente
----------------------	-----------------------------------

- | | |
|---------|------------|
| x += y | x = x + y |
| x -= y | x = x - y |
| x *= y | x = x * y |
| x /= y | x = x / y |
| x %= y | x = x % y |
| x <<= y | x = x << y |
| x >>= y | x = x >> y |
| x &= y | x = x & y |
| x != y | x = x ! y |
| x ^= y | x = x ^ y |



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Opérateurs arithmétiques

Il existe des opérateurs arithmétiques pour les quatre opérateurs fondamentaux et pour calculer le reste d'une division en nombres entiers.

 Syntaxe : *expression1* **op** *expression2*
op  +, -, *, /, %



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

 Opérateurs arithmétiques

Exemples :

Opérateur	Signification	Priorité	Associativité	Exemple
$+$ $-$	Plus unaire Moins unaire	15 15	de gauche à droite de gauche à droite	$+x$ $-x$
$*$ $/$ $\%$	Multiplication Division Modulo(rest. div.)	13 13 13	de gauche à droite de gauche à droite de gauche à droite	$x*y$ x/y $x\%y$
$+$ $-$	Plus binaire Moins binaire	12 12	de gauche à droite de gauche à droite	$x+y$ $x-y$



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Opérateurs arithmétiques Priorité et associativité

Les opérateurs du C++ sont classés dans une échelle de 17 niveaux de priorité où 17 représente la plus haute priorité et 1 la plus basse.

Si un opérateur $op1$ possède un niveau de priorité supérieur à celui d'un opérateur $op2$, une opération avec $op1$ sera effectuée avant une opération à laquelle participe $op2$, si les deux opérateurs apparaissent ensemble dans une expression. Si a , b et c sont des opérandes quelconques, pouvant être traités par $op1$, et $op2$ dans:

$a \ op2 \ b \ op1 \ c$

L'opération $b \ op1 \ c$ sera réalisée en premier, étant donné que $op1$ a un niveau de priorité supérieur à $op2$. En suite le résultat de $b \ op1 \ c$ sera utilisé comme opérande droit de l'opération avec $op2$.

Exemples:

le résultat de l'évaluation de $1+2*3$ est :

le résultat de l'évaluation de $3*3+4/4-5*5$:



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

 Opérateurs de comparaison

Les six opérateurs de comparaison du C++ possèdent deux opérandes et appartiennent donc au groupe des opérateurs binaires.

Opérateur	Signification	Priorité	Associativité	Syntaxe
== !=	Egalité Différent	9 9	De gauche à droite De gauche à droite	expres1 == expres2 expres1 != expres2
> >= < <=	Supérieur Supérieur ou égal Inférieur Inférieur ou égal	10 10 10 10	De gauche à droite De gauche à droite De gauche à droite De gauche à droite	expres1 > expres2 expres1 >= expres2 expres1 < expres2 expres1 <= expres2

Remarque : si l'expression est réalisée (vrai) elle a pour valeur 1 sinon 0

Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Opérateurs logiques

Les trois opérateurs logiques de C++, deux sont binaires et un est unaire.

Opérateur	Signification	Priorité	Associativité	Syntaxe
! (not)	Non logique	15	De droite à gauche	!expression
&& (and)	ET logique	5	De gauche à droite	expr1 && expr2
(or)	OU logique	4	De gauche à droite	expr1 expr2

Remarque : le résultat d'une opération logique a pour valeur 1 ou 0

Exemples: quel est le résultat des expressions suivantes:

- 3 && 7 €
- 1 < 2 && 3 < 4 €
- a = 1, b = 0; a && (b == 2) €
- 3 || 7 €
- x - x || 7 > 3 €
- a = 1 !a €
- a = 0 !a €
- a = 1 A. BEKKHOUCHA ! (a > 1) €



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Opérateurs de bits

Dans les opérateurs bits à bits il existe un opérateur unaire et trois binaires. Ils permettent de manipuler des variables au niveau du bit. Ces opérateurs ne s'effectuent que sur des variables entières (int, long, short, unsigned) et caractères (char et unsigned char).

Opérateur	Signification	Priorité	Associativité	Syntaxe
~	Non binaire	15	De droite à gauche	~expression
&	ET binaire	8	De gauche à droite	expr1 & expr2
^	OU binaire exclusif	7	De gauche à droite	expr1 ^ expr2
	OU binaire inclusif	6	De gauche à droite	expr1 expr2

L'opérateur unaire ~ produit le complément à 1 d'un nombre entier: tous les bits ayant la valeur 1 sont remplacés par des bits avec 0 et inversement.

Exemple: `int i = 240;`
`00000000 11110000` //représentation binaire sur deux octets :
`~i;` // complément de 1 de i donne la valeur `11111111 00001111` soit -241

Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Opérateurs de bits

L'intersection de deux bits avec l'opérateur **&** résulte en un bit **activé** lorsque les deux bits d'origine sont **activés**. Dans tous les autres cas, le bit résultat a toujours la valeur 0. (activé  1)

Exemple: char let1='C', let2 = 'F';

let1	&	let2		0100 0011
				& 0100 0110
				0100 0010

// c'est le code ASCII de la lettre 'B'



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Opérateurs de bits

L'intersection de deux bits avec l'opérateur **|** (ou non exclusif) résulte en un bit **activé** lorsque au moins l'un des deux bits d'origine est **activé**.

Exemple: char let1='C', let2 = 'F';
Let1 | let2 
$$\begin{array}{r} 0100\ 0011 \\ | \ 0100\ 0110 \\ \hline 0100\ 0111 \end{array} \quad // \text{ c'est le code ASCII de la lettre 'G'}$$

L'intersection de deux bits avec l'opérateur **^** (ou exclusif) résulte en un bit **activé** lorsque un seul des deux bits d'origine est **activé**.

Exemple: char let1='C', let2 = 'F';
Let1 ^ let2 
$$\begin{array}{r} 0100\ 0011 \\ ^ \ 0100\ 0110 \\ \hline 0000\ 0101 \end{array} \quad // \text{ c'est le code ASCII 5 de la lettre 'e'}$$



Introduction générale sur les structures de données

Actions élémentaires sur les objets

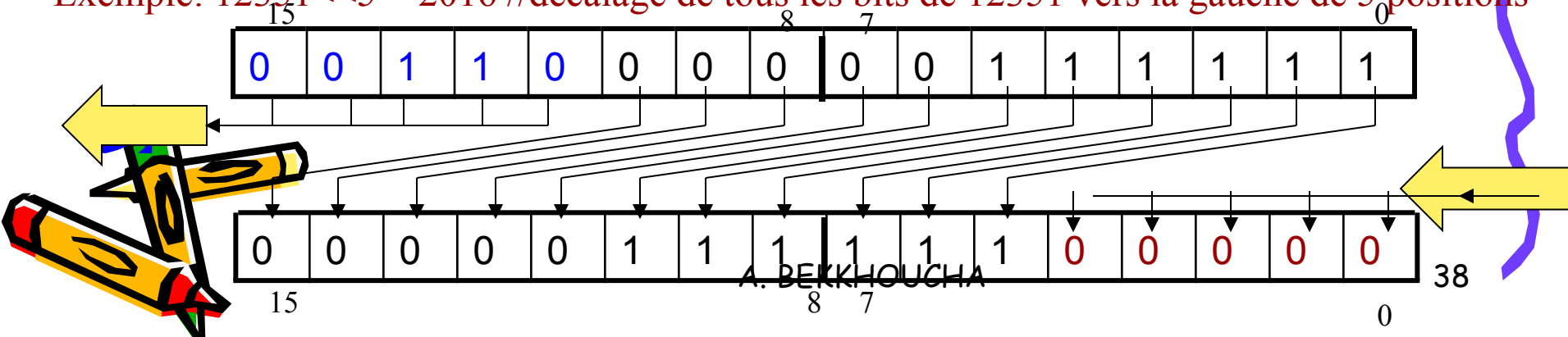
Opérateurs courants en langage C

 Opérateurs de décalage

Les opérateurs de décalage de C++ ont deux opérandes à nombres entiers et décalent les bits de leur opérande gauche vers la gauche ou vers la droite du nombre de positions indiquées dans leur opérande droit. Le résultat de l'opération est du type de l'opération de gauche. Si l'opérande droit est négatif, le résultat de l'opération de décalage n'est pas défini.

Opérateur	Signification	Priorité	Associativité	Syntaxe
<< >>	Décalage vers la gauche Décalage vers la droite	11 11	De gauche à droite	express1<<expres2 expres1>> expres2

Exemple: 12351<<5 = 2016 //décalage de tous les bits de 12351 vers la gauche de 5 positions



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Incrémentation et

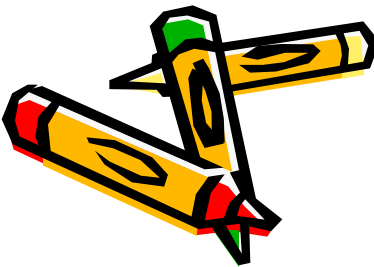
Décrémentation. Il existe deux opérateurs unaires ++ (incrément) et -- (décrément) qui exclusivement à augmenter ou à diminuer de 1 la valeur d'une variable.

Incrément :a++ ou ++a ➦ a=a+1 ou a+=1

Décrément :a-- ou --a ➦ a=a-1 ou a-=1

Opérateur	Signification	Priorité	Associativité	Syntaxe
++	Incrément	15	De droite à gauche	x++(suffixe), ++x(préfixe)
--	Décrément	15	De droite à gauche	x--(suffixe), --x(préfixe)

Exemple: int a, b;
a = 2; // affectation de la valeur 2 à la variable a
a++; // incrément de la variable a de 1: a vaut 3
++a; // incrément de la variable a de 1: a vaut 4
a--; // décrément de la variable a de 1: a vaut 3
--a; // décrément de la variable a de 1: a vaut 2
b= a++; // la valeur de a est affectée à b avant incrément. b vaut 3 a vaut 4
b= --a; // la valeur de a est affectée à b après décrément. b vaut 2 a vaut 3



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Opérateur d'adressage

L'opérateur unaire & fourni l'adresse mémoire d'une variable

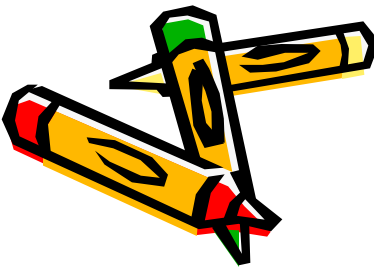
Opérateur	Signification	Priorité	Associativité	Syntaxe
&	Fourni l'adresse mémoire	15	De droite à gauche	&variable

Exemple: `int a = 123;`

la variable `a` contient la valeur 123 alors que `&a` indique l'adresse mémoire de l'emplacement de cette valeur.

`printf("La variable a contient %d.\n Cette valeur se trouve à l'adresse %d", a, &a);`

Le résultat a l'écran est :
La variable a contient 123.
Cette valeur se trouve à l'adresse 12FA



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Les parenthèses

Opérateur	Signification	Priorité	Associativité	Syntaxe
()	Parenthèses de fonction (appel)	16	De gauche à droite	funcX(r, t)
()	Construction d'objet	16	De gauche à droite	classX(p, q)
[]	Indexation de tableau	16	De gauche à droite	Array[Max]
()	Conversion de type explicite (cast)	15	De gauche à droite	(Telttype) x Telttype (x)



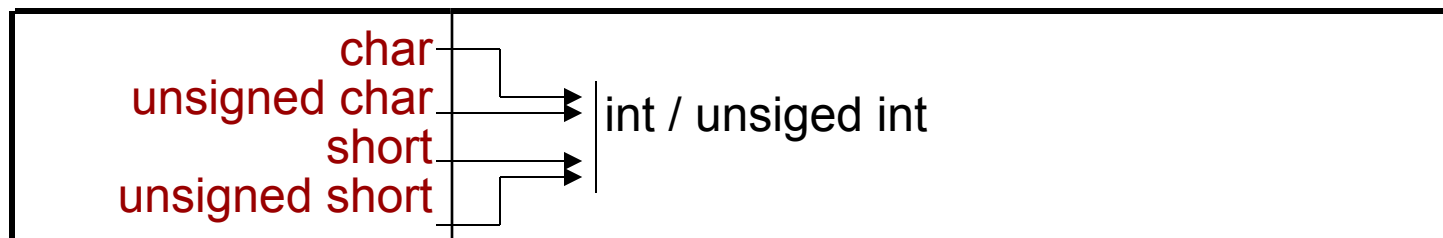
Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Conversion de type

Conversions de type implicites: La promotion entière stipule, dans une opération, les valeurs d'opérandes des types de données **char** et **short** sont toujours transformées en valeurs des types **int** ou **unsigned int**.



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Conversion de type

Conversions de type lors d'opération d'affectation: Si les types des opérandes d'une affectation ne sont pas les mêmes, le type de données de la valeur de l'opérande droit est convertie vers le type de données de l'opérande gauche.

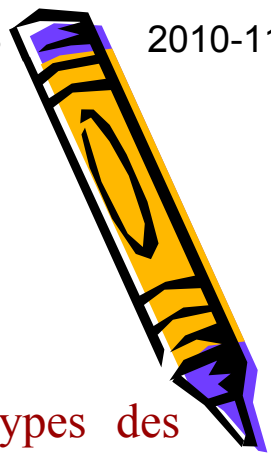
Exemples:

 `int ival=10; double dval; dval = ival; // cela ne pose pas de problèmes,
//transfère du contenu d'une petite variable (2octets) dans une grande (4
octets)`

 `int ival; double dval=1.23; ival=dval; // cela pose problèmes, perte
d'informations`

`//transfère du contenu d'une variable de grande taille (4 octets) dans une
petite (2octets)`

Attention: `float f = 50.0; f=f*1/10 et f*=1/10` ne donneraient pas
le même résultat!!!!!!



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Conversion de type

Conversions de type explicites: La conversion d'un type de données en un autre peut être provoquée explicitement à l'aide de l'opérateur de cast(). Il a la priorité 15, et est utilisable dans ses variantes syntaxiques

 (type) expression // notation traditionnelle

Ftype (expression) // notation fonctionnelle

Dans les deux cas, le type de données de **expression** est transformé dans le type de données **type**, toutefois, la notation fonctionnelle est applicable seulement si **type** est un nom de type simple.

Exemple: (long)123; long(123);
(int)1.234 // entraîne la perte de la partie décimale.



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Conversion de type

Conversions standards de pointeurs et paramètres de fonction:

Les valeurs des paramètres transmises à une fonction lors de son appel sont converties en valeurs d'un autre type lorsqu'elles n'ont pas le type de données qui avait été établi pour les paramètres lors de la définition de la fonction. *Des conversions non standards ont également effectuées pour le type de données pointeur.*



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

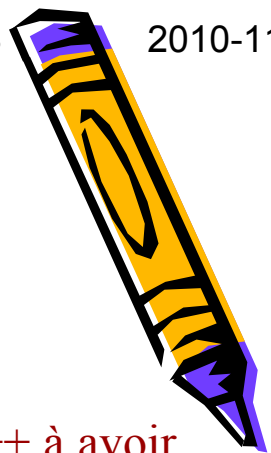
Autres opérateurs

Opérateur de condition: L'opérateur ?: est le seul opérateur de C++ à avoir trois opérandes, il analyse une condition dont la valeur détermine la valeur de l'ensemble de l'expression de l'opérateur conditionnel.

Opérateur	Signification	Priorité	Associativité	Syntaxe
?:	Analyse l'expression de condition	3	De droite à gauche	exp1 ? exp2 : exp3

exp1 représente la condition de l'expression complète. Si cette condition est vrai (non nulle), le deuxième opérande exp2 est analysé, et l'ensemble de l'expression obtient la valeur du deuxième opérande. Si la condition est fausse (nulle) le troisième opérande exp3 est analysé, et l'expression complète obtient la valeur du troisième opérande.

Exemple: `int a;`
`a = 10; a < 0 ? -a: a;`
`a = -10; a < 0 ? -a: a; // équivaut à la fonction valeur absolue`



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Autres opérateurs

L'opérateur de séquence: opérateur virgule ‘,’ permet de réunir deux ou plusieurs expressions sous forme de liste.

Opérateur	Signification	Priorité	Associativité	Syntaxe
,	Crée une séquence d'expression	1	De gauche à droite	exp1 , exp2 , exp3



Introduction générale sur les structures de données

Actions élémentaires sur les objets

Opérateurs courants en langage C

Autres opérateurs

Opérateur sizeof: L'opérateur sizeof donne la taille en octets d'une expression ou d'un type de données et rend ce nombre d'octets en tant que valeur résultante sans préfixe.

Syntaxe: sizeof (expression);
sizeof expression ;

Exemple: char carac; sizeof(carac); //valeur 1
 int entier; sizeof(entier); //valeur 2
 double reel; sizeof(reel); //valeur 8
 sizeof(long) // valeur 4

Autres opérateurs: Il existe des opérateurs utilisés avec les pointeurs et les classes, ils sont:

new, delete, *, ., ->, .*, ->*, ::



Langage C: Gestion des entrées sorties / les sorties

Sorties

Conventionnelles:

Pour accéder aux fonctions d'entrées-sorties, il faut inclure dans le programme le fichier d'en-tête `stdio.h`

```
#include <stdio.h> // en-tête pour entrées-sorties standard
```

Sorties formatées

syntaxe:

```
printf(chaîne formatée, [param1, param2, param3, ...])
```

valeurs ou variables 

Exemples

```
printf("%d", 123); ----> sortie à l'ecran : 123
```

```
printf("%d , %d, et %d sont des nombres entiers", 1, 2, 3);
```

```
----> sortie à l'ecran : 1 , 2, et 3 sont des nombres entiers
```

A. BEKKHOUCHA

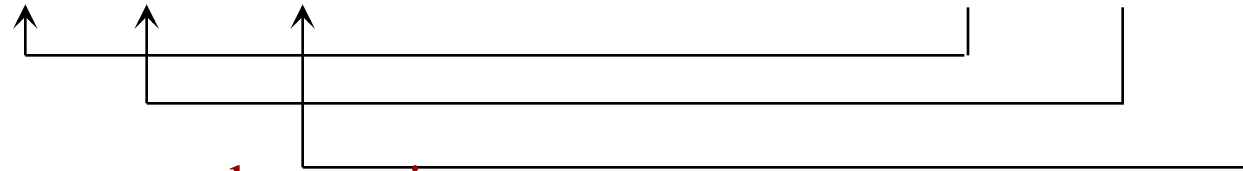
Langage C: Gestion des entrées sorties / les sorties

Sorties Conventiennelles:

```
printf("%d , %d , et %d sont des nombres entiers", 1, 2, 3);
```

----> sortie à l'ecran : 1 , 2 , et 3 sont des nombres entiers

```
printf("%d , %d , et %d sont des nombres entiers", 1 , 2 , 3 );
```



Format pour un nombre entier:

%d, %i nombre décimal de type entier

%hd , %hi nombre décimal de type short

%ld , %li nombre décimal de type long

%o, %ho, %ol nombre octal de type respectivement int short et long

%x, %X; %hx, %hX; %lx, %lX nombre hexadecimal de type int , short et long

%u, %hu, %lu nombre décimal de type unsigned int, short, long

%c un caractère

Langage C: Gestion des entrées sorties / les sorties

Sorties

Conventionnelles:

Base de numérotation: s'effectue à l'aide du # placé entre le % et la lettre du format

Exemple:

```
printf("sans mention de la base: \t%o\t%x\t%X", 123,123,123);  
printf("avec mention de la base: \t%#o\t%#x\t%#X", 123,123,123);
```

l'affichage à l'écran donne:

```
sans mention de la base: 123 7b 7B  
avec mention de la base: 0123 0x7b 0X7B
```

Spécification du signe: s'effectue à l'aide du + placé entre le % et la lettre du format même pour les nombres positifs

exemples:

```
printf("%+d",1) donne à l'écran:+1  
printf("%d",1) donne à l'écran: 1  
printf("+%d",1) donne à l'écran:+1 // le plus + est ici un caractère mais pas un signe
```



Langage C: Gestion des entrées sorties / les sorties

Sorties

Conventionnelles:

Largueur d'édition: spécifiée par un nombre entier inséré dans le spécificateur de format.

```
Exemple: printf( "%d\n"          // largeur par défaut
                "%2d\n"          // Largeur insuffisante
                "%8d",           // Largeur excessive
                1234,1234,1234);
```

donne à l'écran:

1234

1234

1234 // quatre blancs sont insérés avant le 1234

Caractère de remplissage: spécifié par un caractère inséré dans le spécificateur de format.

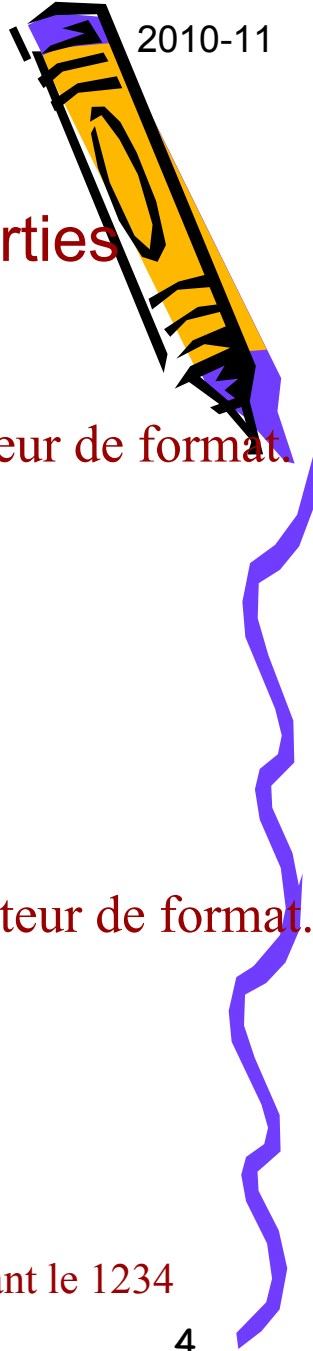
```
Exemple: printf( "%d\n"
                "%2d\n"
                "%08d",          // Caractère de remplissage 0
                1234,1234,1234);
```

donne à l'écran:

1234

1234

00001234 // quatre caractères 0 sont insérés avant le 1234



Langage C: Gestion des entrées sorties / les sorties

Sorties

Conventionnelles:

Nombre de chiffres affichés:

Exemple: `printf( "affichage de \n%8.6d", 23);` // Largeur de l'affichage :8. Nombre de chiffres affichés:6
donne à l'écran:

affichage :

000023 // Deux blancs et quatre zéros sont insérés avant le 23

Alignement à droite et à gauche:

Remarque: l'alignement est par défaut à droite. Afin de le modifier il faut mettre un "-" dans le spécificateur de format.

Exemple: `printf( "affichage à droite %8d* par défaut\n affichage à gauche :%-8d*", 23,23);`

donne à l'écran:

affichage à droite: 23* par défaut
affichage à gauche:23 *





Langage C: Gestion des entrées sorties / les sorties

Sorties

Conventionnelles: Spécificateur de Format pour un nombre à virgule flottante:

Spécificateur de format	Valeur affichée
%f	Nombre flottant de type float ou double
%e, %E	"
%g, %G	"
%Lf, %Le, %LE, %Lg, %LG	Nombre flottant de type long double

Exemple:

instruction d'affichage	Résultat à l'écran
printf("%f", 1.234);	1.234000
printf("%e %e %e", 0.1234,1.234,12.34);	1.234000e-001 1.234000e+000 1.234000e+001
printf("%g %g %g",123.0, 123., 123.4);	123 123 123.4
printf("%g", 1.234e2)	123.4
printf("%g %G",0.0000123,1234567.0);	1.23e-005 1.234567E-006

Remarque: les spécificateurs %g et %G entraînent une représentation dans le format exponentiel lorsque l'exposant normalisé est inférieur à -4 ou supérieur au nombre de chiffres requis (6)



Conventionnelles:

Langage C: Gestion des entrées sorties / les sorties

Sorties

Conventionnelles: Spécificateur de Format pour une chaîne de caractères:

format:

<code>printf("%s","une chaîne");</code>	---->	une chaîne	
<code>printf("une chaîne");</code>	---->	une chaîne	// le même résultat qu'avant

Largeur d'affichage:

<code>printf("%12s", "une chaîne");</code>	---->	* une chaîne*
--	-------	------------------

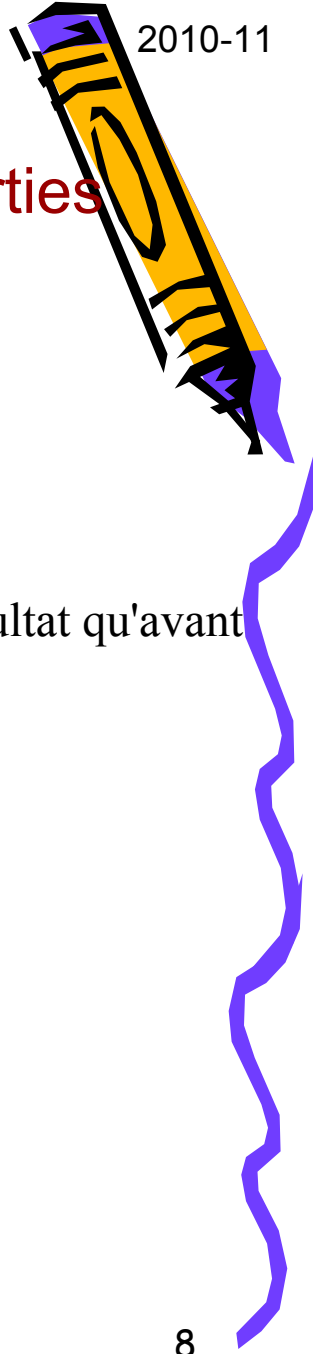
Alignement:

<code>printf("%-12s", "une chaîne");</code>	---->	*une chaîne *
---	-------	------------------

Nombre maximal de caractères à afficher:

```
printf("%6.1s\n%6.2s\n%6.3s\n %6.4s\n %6.5s\n %6.6s",  
      "chaîne","chaîne","chaîne","chaîne","chaîne","chaîne",);
```

c
ch
cha
chaî
chaîn
chaîne



Langage C: Gestion des entrées sorties / les sorties

Sorties

Conventionnelles:
Spécificateur de Format pour une adresse :

Exemple:

```
int i;  
printf("%p", &i);           // affiche l'adresse de la variable i  
// le résultat est généralement un nombre hexadécimal comme 0XB7A
```

Autre exemple:

```
printf("%X \t %d",&i, &i)  ---> B7A    2938  
      formes      hexadécimal      decimal
```

Format %n :

Exemple: `int compt; //détermination du nombre de caractères jusqu'au !`

```
printf("Jusqu'au caractère !%n il y a", &compt);  
printf("%d caractères", compt);  
résultat --->Jusqu'au caractère ! il y a 19 caractères
```



Langage C: Gestion des entrées sorties / les Entrées

Entrées Conventionnelles:

Entrées Formatées:

syntaxe:

```
scanf("chaîne formaté, [adr1, adr2, adr3, ...])
```

adresses de variables

Exemples:

1- int variable;

```
scanf("%d", &variable); //Lecture d'un entier au clavier et le mémorise dans "variable"
```

2- int var1, var2, var3;

```
scanf("%d %d %d", &var1, &var2, &var3);
```

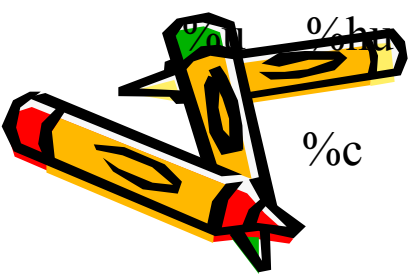
Lecture de trois entiers au clavier et les mémorise dans les variables var1, var2, var3

Langage C: Gestion des entrées sorties / les Entrées

Entrées Conventionnelles:

Spécificateur de format pour des nombres entiers

Spécificateur de format	Valeur affichée
%d %hd %ld	Nombre <u>décimal</u> de type int short long respectivement
%i %hi %li	Nombre <u>entier</u> de type int, short, long respectivement La valeur saisie peut être décimale, octal (0) ou hexadécimal (0x)
%o %ho %lo	Nombre <u>octal</u> de type int, short, long respectivement
%x %hx %lx %X %hX %lX	Nombre <u>hexadécimal</u> de type int, short, long respectivement
%hu %lu %c	Nombre <u>hexadécimal</u> de type unsigned int, unsigned short, unsigned long respectivement caractère



Langage C: Gestion des entrées sorties / les Entrées

Entrées Conventionnelles:

Spécificateur de format pour des nombres à virgule flottante

Spécificateur de format	Valeur affichée
%f %lf %Lf	Nombre <u>flottant</u> de type float , double , long double respectivement
%e %le %Le %E %lE %LE	Nombre <u>flottant</u> de type float , double , long double respectivement (notation exponentielle)
%g %lg %Lg %G %lG %LG	Nombre <u>flottant</u> de type float , double , long double respectivement (notation exponentielle ou virgule flottante)



Langage C: Gestion des entrées sorties / les Entrées

Entrées Conventionnelles:

Autres spécificateurs de format :

Nombre de caractères tapés:

long valeur; int compt; //declaration des variables

scanf("%ld%n ",&valeur, &compt); // la variable "compt" contient le nombre de caractères tapés

Ignorer des caractères:

le caractère "*" inséré dans le format de lecture permet d'éviter le nombre lu.

Exemple: scanf("%d %*d %d",&var1,&var3); La saisie des nombres 1 2 3 permet de mémoriser le 1 et le 3 tout en ignorant le 2

Largeur d'un champ: la largeur d'un champ est insérée dans le format de saisie.

scanf("%1d %2d %3d",&var1,&var2,&var3);

La saisie de 123456 permet de mettre le 1 dans var1, le 23 dans var2 et 456 dans var3 sans séparer ces nombres d'espaces

Langage C: Les fonctions

Un programme C peut être structuré et subdivisé en petites unités appelées fonctions.

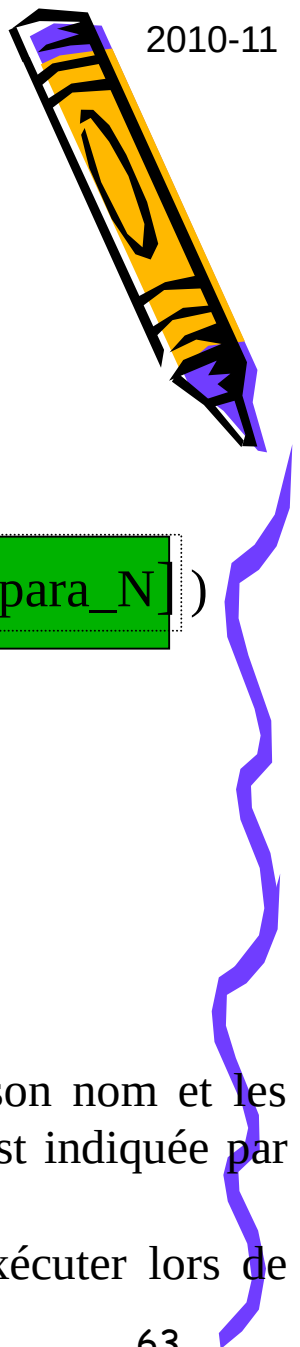
Une fonction est une section remplissant une tâche partielle du programme global.

Une fonction est réutilisable aussi souvent que possible: gain de temps d'écriture de code, réutilisation dans différent endroit du programme.



Langage C: Les fonctions - déclaration

Définition et appel de fonctions



Définition d'une fonction:

Type_valeur_retournée Nom_Fonction ([type1 para_1, ...,typeN para_N])

Liste des paramètres

{
[Ensembles des instructions]
}

corps de la fonction

La fonction se divise en deux parties:

- l'entête qui indique le type de la valeur retournée par la fonction, son nom et les paramètres entre parenthèses. Une fonction ne retournant pas de valeur est indiquée par void
- le corps de la fonction est le bloc qui contient les instructions à exécuter lors de l'appel de la fonction.



Langage C: Les fonctions - déclaration

Définition et appel de fonctions

Appel d'une fonction:

```
Nom_Fonction([para_1,...,para_N]);
```

Les fonctions se définissent en un endroit quelconque du programme, sauf à l'intérieur d'une autre fonction.

Toute fonction doit être **connue** du compilateur avant son appel.



Langage C: Les fonctions - déclaration

Déclaration des fonctions (prototypes)

[Type_valeur_retournée] Nom_Fonction_([type1 [para_1], ...,
typeN [para_N]]) ;

Liste des paramètres

Exemples de déclarations par prototypes:

- 1- **void** Ma_fonction (**int** debut , **int** fin) ;
- 2- **void** Ma_fonction (**int** , **int**);

La déclaration de la fonction est une simple présentation de celle-ci.

Une même fonction peut être déclarée aussi souvent que nécessaire, alors qu'elle ne peut être définie qu'une seule fois.



Langage C: Les fonctions - déclaration

Déclaration des fonctions (prototypes)

Déclaration locales et globales

La déclaration d'une fonction est **globale** si elle est faite en dehors de toute fonction.

Exemple:

```
#include <stdio.h>
int Ma_fonction( )
{ // corps de la fonction
}
void Ma_deuxieme_fonction( )
{ // corps de la fonction
  int val_entiere = Ma_fonction( );
```

```
void main( )
{ // programme principal
  // déclaration et initialisation de
  la
  // variable par appel de la fct
  int valeur = Ma_fonction( );
  // appel de la fct
  Ma_deuxieme_fonction( );
  ....
}
```

Langage C: Les fonctions - déclaration

Déclaration des fonctions (prototypes)

Déclaration locales et globales

La déclaration d'une fonction dans une autre fonction est qualifiée de **locale**.

Exemple:

```
#include <stdio.h>
void main( )
{
    // programme principal
    int Ma_fonction ( );           // déclaration de la fonction
    void Ma_deuxieme_fonction( ); // déclaration de la deuxième fonctions
    int valeur = Ma_fonctio( );    // appel de la fonction
    Ma_deuxieme_fonction( );       // appel de la fonction
}

int Ma_fonction( )                // spécification de la fonction
{ // corps de la fonction }

void Ma_deuxieme_fonction( )
{ // coprs de la fonction
    int val_entiere = Ma_fonction( ); //ERREUR Ma_fonction n'est pas connue ici
}
```

28/09/2025

A. BEKKHOUCHA

67

Remarque: La déclaration de «Ma_deuxieme_fonction() » dans le P.P est implicite dans « Ma_fonction »

Langage C: Les fonctions - déclaration

Déclaration des fonctions (prototypes)

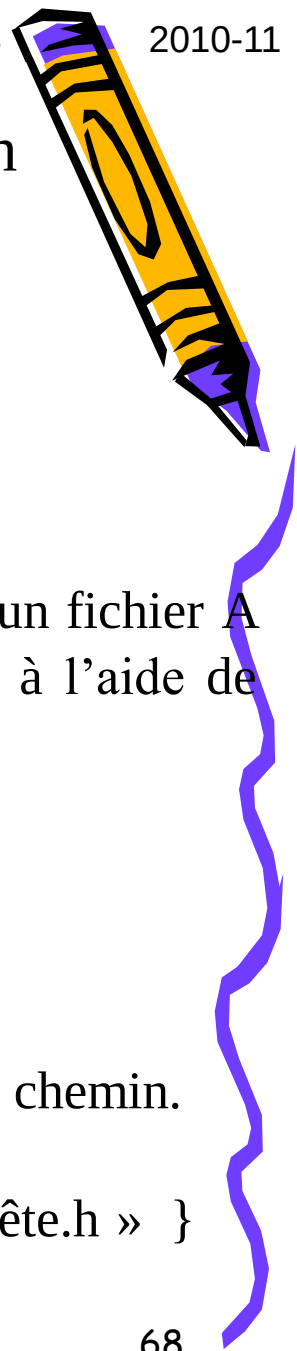
Déclaration de fonctions dans des fichiers d'en-tête

Il est possible de déclarer et de définir une fonction dans un fichier A et de l'utiliser dans un autre fichier B à condition de la déclarer à l'aide de l'inclusion d'un fichier entête.

Exemple:

```
#include <stdio.h> // déclaration de la librairie standard
#include "fichier_entete.h" // déclaration du fichier en tête.
                        // Si nécessaire il faut indiquer le chemin.

void main( )
{ // appel des fonctions déclarées dans le fichier «fichier_entete.h » }
```



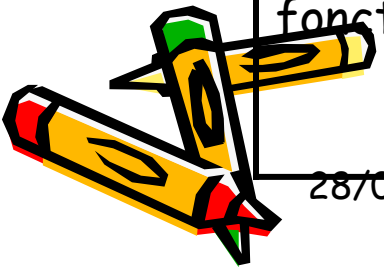
Langage C: Les fonctions- les variables

Domaine de validité des variables ou objets



Tout objet, variable ou fonction, selon l'emplacement de sa déclaration ou sa définition, possède une certaine portée, domaine dans lequel il est utilisable et valide.

Variable globale	Variable locale
Un objet est déclaré global si sa déclaration est effectuée en dehors de tout bloc et de fonction.	Lorsqu'un objet est déclaré dans un bloc ou fonction, sa portée débute avec cette déclaration ou définition, et elle se limite au bloc ou la fonction.



Langage C: Les fonctions- les variables

Domaine de validité des variables: **homonyme** et **masquage**

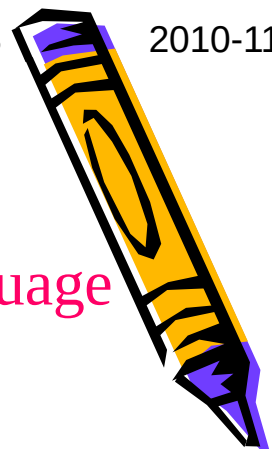
Variables de même nom:

Il est possible de donner le même nom à deux variables définies dans deux blocs différents

Exemple:

```
void fonct( ); // déclaration de la fonction
void main ( )
{ int a = 1 ;      // variable locale. Portée :main
  printf("%d", a); // affichage du contenu de 'a' dans le main
  fonct( );       // appel de la fonction
} // fin du bloc main
```

```
void fonc( )
{ int a = 2;      // déclaration de la variable locale
  printf("%d", a); // affichage du contenu de la variable locale à la fonction
}
// donner le résultat de l'exécution de ce programme!
```



Langage C: Les fonctions- le 'return'

L'instruction '**return**' termine l'exécution de la fonction en retournant une valeur de même type que celui de la déclaration de la fonction. Si une fonction ne retourne pas de valeur, elle est déclarée de type **void**.

Exemple

Fonction retournant une valeur

```
int fonct_entière( )  
{ int val;  
.....  
return(val); // met fin à l'exécution  
              // de la fonction
```

Fonction **sans** retour de valeur

```
1°) void fonct_sans_valeur( )  
    { .....// bloc d'instructions }  
  
2°) void fonct_sans_valeur( )  
{  
    .....// bloc d'instructions  
    return;
```

Langage C: Les fonctions- les paramètres

Transmission par valeur

La transmission par **valeur** d'un paramètre implique qu'à l'appel de la fonction une **copie** de la **valeur** du paramètre soit réalisée et **initialise** le paramètre **formel** correspondant, sauf pour les tableaux. La fonction n'a aucun accès à leur emplacement mémoire et **qu'elle ne peut donc pas les modifier.**

Exemple:

```
int somme(int debut, int fin)
{ int total = 0;
  while(debut<=fin)    total +=
  debut++;
  /* la variable debut est modifiée
  localement dans la fonction
  'somme'*/
}
```

```
void main( )
{ int initiale, finale;
  int la_somme;
```

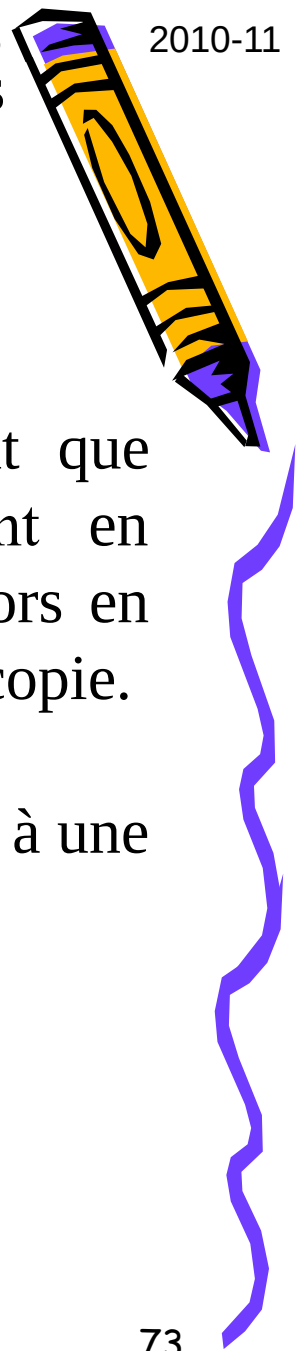
```
initiale = 10; finale = 20;
La_somme = somme( initiale,
finale);
printf("la somme vaut %d",
      la_somme);
printf("initiale vaut %d", initiale);
}
/*Deviner le résultat de
l'exécution de ce programme*/
```

Langage C: Les fonctions- les paramètres

Transmission par **adresse**

L'adresse d'un 'objet' pourra être transmise en tant que paramètre à l'appel d'une fonction. L'emplacement en mémoire de la donnée étant connu, la fonction est alors en mesure d'accéder à l'objet proprement dit et non à une copie.

L'objet passé, par adresse, en tant que paramètre formel à une fonction, peut être de type pointeur ou de type simple.



Langage C: Les fonctions- les paramètres

Transmission par **adresse**

Exemples:

```
void somme(int debut, int fin, int
*total)
{int var_locale = 0;
  while(debut <= fin)
    var_locale += debut++;
  *total = var_locale;
}
```

```
void main()
{int la_somme;
// paramètre de type simple
somme(10, 20, &la_somme);
printf("la somme vaut d", la_somme);
}
```

28/09/2025

```
void somme(int debut, int fin, int
*total)
{int var_locale = 0;
  while(fin <= debut)
    var_locale += debut++;
  *total = var_locale;
}
```

```
void main()
{int *pt_somme;
  pt_somme = (int *)malloc(sizeof(int));
// paramètre de type pointeur
somme(10, 20, pt_somme);
printf("la somme vaut %d",
*pt_somme);
}
```

A. BEKRI



Langage C: Les fonctions- les paramètres

Transmission par **adresse**

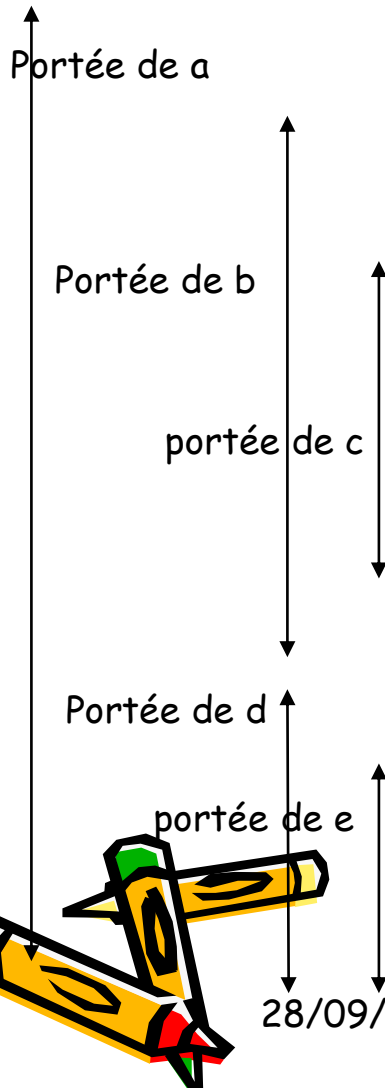
Tableaux en tant que paramètres

Un Tableau ne peut exclusivement être transmis que par adresse, ce qui signifie que la fonction appelée ne travaille jamais sur une copie locale du tableau et que chaque modification dans une fonction citant ce paramètre formel change le tableau d'origine.

Syntaxe et exemple:

```
void fonction( int proto[10] )  
{ ...}  
void main()  
{int Le_Tableau[10];  
  ...  
  fonction(Le_Tableau); // appel par adresse du tableau  
  ..  
}
```

Langage C: Variables globales et variables locales



```
#include <stdio.h> //lib. standard
void fonct( ); // déclaration de la fonction
int a=1; //déclar. et initialis. de la variable globale 'a'
void main ( ) //programme principal
{int b=2 //déclar. et initialisation de la variable locale 'b'
  printf("%d", a ); // accès à la variable globale.
  printf("%d", b ); // accès à la variable locale.
  { //début de bloc
    int c=3; //déclar. et initialis. de variable locale au bloc
    printf("%d", a ); //accès à la vari. globale dans le bloc.
    printf("%d", b); //accès à la variable locale dans le bloc.
    printf("%d",c); //accès à la vari. locale du bloc dans le main.
  } // fin de bloc
  fonct( ); // appel de la fonction fonct
} //fin du main
int d = 4; // déclaration de la variable globale 'd'
void fonct( )// définition de la fonction
{ int e =5; // déclaration de la variable locale à la fonction
  printf("%d %d %d", a, d, e);
  // accès aux variables globales 'a' et 'd' et à la variable locale 'e'
} // fin de la définition de la fonction
```

28/09/2025

A. BEKKHOUCHA

Les tableaux

Tableaux a une dimension

type Nom_du_Tableau [Nombre_Elements];

Exemples:

```
int      Tab_int[10]; /* Un tableau de 10 entiers. 10 fois 2 octets*/  
float    Tab_float[20];  
char     Tab_char[30];
```

Affectation:

```
Tab_int[5] = 125; /*affectation de la valeur 125 au 6ème élément du tableau */  
c = Tab_char[20];  
/* Le contenu de l'élément indexé par 20 est affecté à la variable c */
```



Les tableaux

Tableaux a deux dimensions

Syntaxe par l'exemple:

```
int      Tab2_int[10][10];          /* Tableau à deux dimensions */
float    Tab2_float[10][20];/* Tableau à deux dimensions */
char     Tab3_char[2][3][4];/* Tableau à trois dimensions */
```

* Initialisation de tableaux

Au moment de la déclaration des tableaux, il est possible de procéder en même temps à leur initialisation.

Exemples:

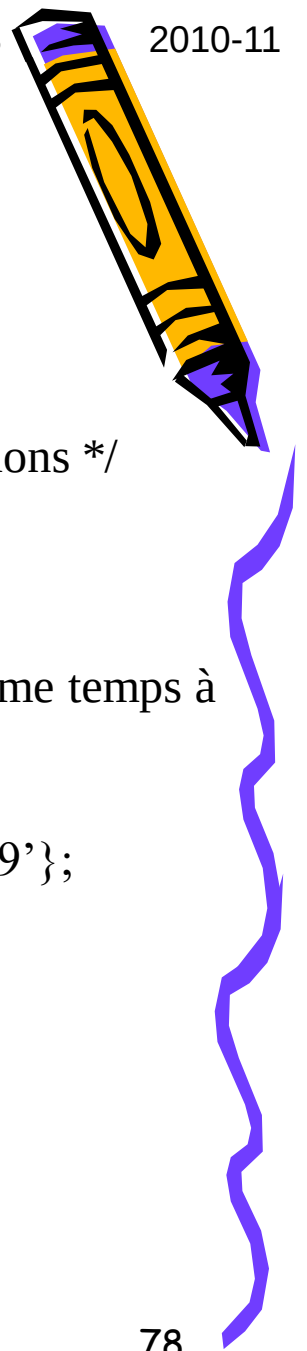
```
char     Tab_10_caractères[10] = {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'};
```

```
int      Tab3_car[2][3][4] = { { {000, 001, 002, 003},
                                {010, 011, 012, 013},
                                {020, 021, 022, 023},
                                },
                                { {100, 101, 102, 103},
                                {110, 111, 112, 113},
                                {120, 101, 122, 123},
                                },
                                };
```

28/09/2025

A. BEKKHOUCHA

78



Les tableaux

Tableaux a deux dimensions

Lors de l'initialisation une dimension peut ne pas être fixée, le compilateur la fixera automatiquement en fonction du nombre de valeurs d'initialisation.

Exemples :

```
char Voyelles []= "AEIUOY" /* Tableau à 6 éléments */
```

```
int Tab[2][ ] =      { {1 , 2 , 3 , 4} ,  
                      {5 , 6 , 7 , 8}  
                      };
```

```
int Tab2[][4] =      { {1 , 2 , 3 , 4} ,  
                      {5 , 6 , 7 , 8}  
                      }; /* Déclaration illégale */
```

Tableau à plusieurs dimensions

En C, les tableaux peuvent avoir plusieurs dimensions.
Ils sont considérés comme des tableaux de tableaux.



Les chaînes de caractères

Syntaxe

"Ceci est une chaîne de caractères entre deux guillemets"

Les chaînes de caractères sont représentées en C sous forme d'un vecteur de caractères se terminant par un caractère spécial qui est le caractère NULL ('\0'). La gestion de ce caractère terminal est à la charge du programmeur. Lorsque vous réservez de la place pour une chaîne, il convient de tenir compte de ce caractère et donc de rajouter 1 octet pour la place nécessaire.



Les structures

Syntaxe:

```
struct  Nom_structure{  
    type_1  Champ1;  
    type_2  Champ2 , Champ3 ;  
    .....  
    type_N  ChampN;  
} Nom_var1 , ....., Nom_varp;
```

Tous les champs d'une structure sont alloués en mémoire en un seul bloc; chaque champ étant implanté dans l'ordre de la déclaration et occupant la place dévolue à son type.



Les structures

Exemple:

Soit la structure de données relative aux abonnés au téléphone définie par:

- Nom de l'abonné /* chaîne de 20 caractères */
- Prénom /* chaîne de 20 caractères */
- Adresse:
 - Numéro de la rue/* un entier*/
 - Nom de la rue /* chaîne de 40 caractères */
 - Nom de la ville /* chaîne de 20 caractères */
 - Code postal /* un entier */
- Numéro de téléphone:
 - Indicatif de région /* un entier */
 - Numéro de téléphone /* 3 entiers*/



Les structures

Accès aux champs d'une structure:

Seuls les champs d'une structure peuvent faire l'objet d'opérations. Ceux-ci appartenant à une variable structure, il est nécessaire de disposer d'un opérateur permettant de signifier que tel champ appartient à telle structure. Cet opérateur est le '.' (point).

Exemples:

```
Nom_var1.champ1 = valeur_de_même_type_que_champ1;
```

```
variable_de_même_type_que_champN = Nom_var1.champN
```

Comment l'accès se fait à un champ d'une structure incluse dans une autre structure?



Les unions



Syntaxe:

```
union  Nom_Union{
    type_1  Champ1;
    type_2  Champ2 , Champ3 ;
    .....
    type_N  ChampN;
} Nom_var1 , ..... , Nom_varp;
```

Une **union** est un ensemble de champs occupant tous le même emplacement en mémoire. Les champs d'une union commencent tous à la même adresse. L'occupation mémoire est donc déterminée par le plus important des champs.

Exemple:

```
Union Exple {  int i;                /* 2 octets */
                float somme;          /* 4 octets */
                double resultat;     /* 8 octets */
                char chaine[10];     /* 10 octets*/
                } Mon_union;
```

La taille de "Mon_union" est deoctets



Les champs de bits

Syntaxe:

En C, il est possible de manipuler l'information au niveau du bit. Pour ce faire, on procède via une structure de champs de bits. A la suite de la déclaration des champs de la structure, il convient de préciser la taille en bits du champs. Celle-ci séparée du nom de champ par l'opérateur ':':

Exemple:

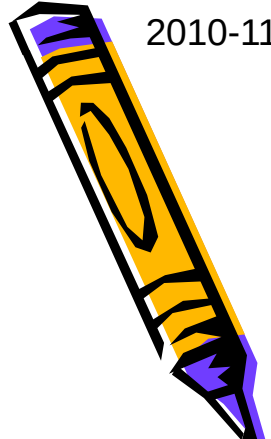
```
struct champ_de_bits
{
    unsigned champ1_bits : 3 ;
    unsigned champ2_bits : 2 ;
    unsigned champ3_bits : 7 ;
};
```

Remarques:

- La taille d'un champ ne peut dépasser 15 bits,
- L'ensemble est traité par mot machine (16bits),
- Allocation des champs s'effectue de droite à gauche.



Les pointeurs



Syntaxe de déclaration de pointeurs:

type *var_pt ;

type : désigne un type prédéfini ou défini par l'utilisateur de type simple ou structuré.

* : l'étoile indique qu'il s'agit d'un pointeur.

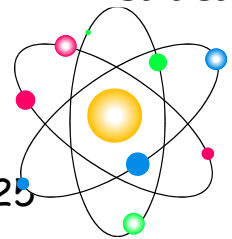
var_pt : Le nom de la variable.

Cette déclaration signifie que la variable "var_pt" est un pointeur sur "type".
Exemples:

int	*pt_int;
char	*pt_char;
struct	abonné *pt_abonne;

Ces déclarations réservent uniquement la place mémoire des variables pointeurs spécifiées, pas celle des zones qu'ils pointeront.

A. BEKKHOUCHA



28/09/2023

Les pointeurs

Allocation de la place mémoire pour un pointeur:

Pointeurs et variables simples

Exemples: /* déclaration de variables */

```
int i, *pt_int;  
char c, *pt_char;
```

La déclaration des variables "simples" 'i' et 'c' réserve de la place mémoire pour chacune de ces variables dont les adresses sont données par '&i' et '&c'. Il est possible de faire un "branchement" des pointeurs '*pt_int' et '*pt_char' sur les adresses des variables simples par les opérations d'affectation suivante:

```
pt_int = &i;  
pt_char = &c;
```

Les variables simples et les variables pointées, de même type, désignent le même emplacement mémoire. Donc le contenu de la variable i (respectivement c) et celui de *pt_int (respectivement *pt_char) est le même.

Exemple :

```
i = 12548;          pt_int = &i;  
c = 'a';            pt_char = &c;  
printf("le contenu de i vaut %d\n",i);  
printf("le contenu de pt_int vaut %d\n", *pt_int);  
printf("le contenu de c vaut %c\n",c);  
printf("le contenu de pt_char vaut %c\n", *pt_char);
```

28/09/2025

A. BERKHOUCHE



Les pointeurs

Pointeurs et tableaux

Un tableau se déclare en donnant son type, son nom et le nombre d'éléments. Le nom d'un tableau est en fait à considérer comme un pointeur constant sur le premier élément de celui-ci, l'élément numéro zéro.

Exemple :

```
int Tab_int[10]; /* déclaration d'un tableau de 10 entiers */
```

`Tab_int[0]` **est le premier élément du tableau.**

`Tab_int` **est un pointeur qui pointe sur le premier élément du tableau.**

Si nous définissons en plus un pointeur d'entier `int *Ptr`, nous pouvons affecter à `Ptr` l'adresse de début de tableau, comme suit:

```
Ptr = &Tab_int[0];
```

ou bien

```
Ptr = Tab_int;
```

Les éléments d'un tableau sont accessibles de deux façons différentes:

`Tab_int[i]` -----> accès direct par l'indexe.

`*(Tab_int + i)` -----> accès par pointeur.

Pour passer à l'élément suivant il suffit d'incrémenter le pointeur de 1.



Les pointeurs

Pointeurs et structures

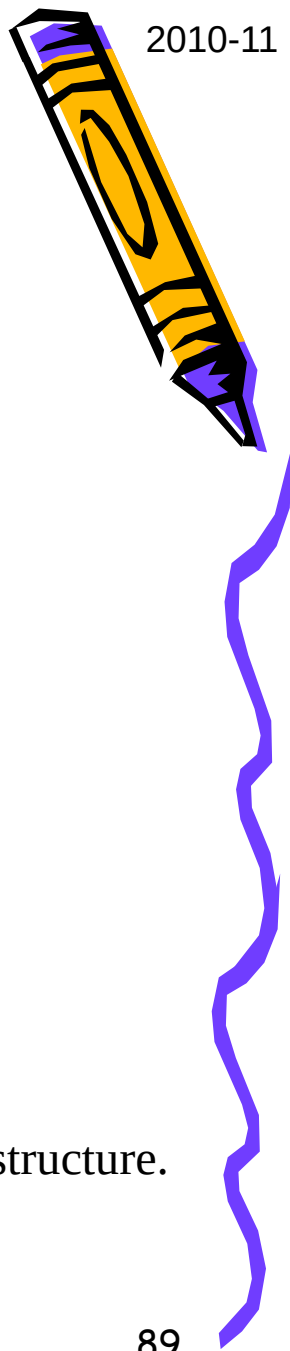
Il est possible de déclarer des pointeurs pointant sur une structure.

Exemple:

```
struct Abonne
{
    char Nom[20];
    char Prenom[20];
    int numero;
    .....
    .....
    .....
} Client , *Ptr_client;
```

Client est une variable simple de type structure;

Ptr_client est une variable de type pointeur qui pointe sur une zone structure.



Les pointeurs

Pointeurs et structures

- Accès aux zones structures pointées

Un pointeur ne peut être utilisé sauf s'il a une adresse d'une zone "correcte". Pour cela nous faisons pointer *Ptr_client* sur la même zone que celle de la variable *Client*.

L'accès à un champ d'une structure pointée par une variable de type pointeur sur l'exemple ci-dessus suit la syntaxe suivante:

```
Ptr_client->numero = 12;  
Ptr_client->Nom[0] = 'B';
```

La "flèche" `"->"` indique l'accès d'un champ d'une structure pointée par une variable de type pointeur.



Les pointeurs

Pointeurs et chaînes de caractères:

-Allocation statique

La déclaration d'un tableau de caractères est équivalente à une chaîne de caractères avec une réservation statique de la place mémoire. Cette place est délimitée par le nombre d'éléments du tableau ou par le nombre de caractères lors de l'initialisation par déclaration.

Exemple:

```
char Tab_char[80]; /* un tableau de 80 caractères. Un string de 79 caractères*/  
char Nom[ ] = "Abou Qassim";
```

Le "remplissage" du tableau *Tab_char* s'effectue soit caractère par caractère ou à l'aide des fonctions de manipulation de chaîne de caractères.

```
Tab_char = "Inittialisation du tableau" /* Ceci est une déclaration illégale */  
strcpy(Tab_car, "Ceci est une initialisation correcte");
```



Les pointeurs

Pointeurs et chaînes de caractères:

-Allocation dynamique

Syntaxe des fonctions d'allocation dynamique de la mémoire:

```
Nom_var_ptr = (Type_var *)malloc(taille_souhaitée);
```

```
Nom_var_ptr = (Type_var *)calloc(nombre_élément, taille_élément);
```

```
void (Type_var *)realloc(Nom_var_ptr, nouvelle_taille_souhaitée);
```

Les tailles sont données en octets. Ces fonctions retournes l'adresse du premier octet. La fonction calloc demande une allocation de *nombre_élément* blocs chacun ayant comme taille *taille_élément* .

S'il n'y pas assez de place, l'allocation ne c'effectue pas et le pointeur reçoit la valeur NULL.



Les pointeurs



Pointeurs et chaînes de caractères:

Allocation dynamique

Exemples:

```
#include <alloc.h>      /* il est nécessaire de faire appel à ce fichier externe pour une
utilisation des fonctions d'allocation de mémoire Pour certain compilateur le fichier à inclure est
malloc.h*/
```

```
int *ptr_int;
char *ptr_c;
```

Avant toute utilisation des variables de type pointeur, il faut leur réserver de la place soit de façon statique, en faisons un branchement sur un autre variable, soit dynamique à l'aide des fonctions d'allocation de la place mémoire.

```
ptr_int = (int *)malloc(2);      ou bien ptr_int = malloc(sizeof(short int));
*ptr_int =1245;
ptr_c = (char *)calloc(20, sizeof(char));    /*ce qui correspondrais à un tableau de 20
caractères*/
```

```
strcpy(ptr_c, "chaîne de caractères");
si la place s'avère insuffisante, elle peut être augmentée de 5 par:
realloc(ptr_c, 25);
```



Les Structures de Données



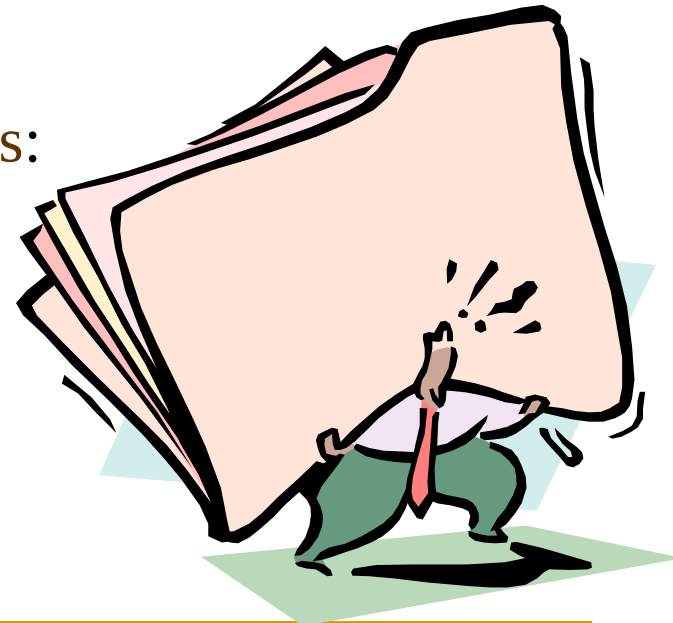
Plan

Structures de données statiques:

- 1 Définition
- 2 Structure de données homogènes
- 3 Recherche

Structures de données dynamiques:

- 1- les structures dynamiques
- 2- les listes linéaires
- 3- les piles
- 4- les files



Les Structures de Données

Définition

Une structure de données est un ensemble organisé d'informations reliées logiquement. Ces informations peuvent être traitées collectivement ou individuellement. Les traitements peuvent porter sur chaque composante prise individuellement ou dans certain cas globalement. Une structure de données est caractérisée par ses **composantes** et leur **arrangement** mais surtout par son **mode d'utilisation**. En effet, les structures de Pile et de File, que nous allons aborder par la suite, peuvent avoir les mêmes composantes, les mêmes arrangements mais elles sont considérées comme différentes car leurs **modes d'exploitations** et **d'utilisations** sont fondamentalement **différents**.

Les Structures de Données

Spécification d'une structure de données

Spécification fonctionnelle:

Exemple tirés du domaine informatique ou de la vie courante.

Mettre en évidence les propriétés générales et les opérations caractéristiques de la structure.

Spécification logique:

Correspond à la description logique des données et des algorithmes de manipulation.

Spécification logique d'une structure comporte deux parties :

La spécification morphologique : définition d'un type complexe pour représenter la structure considérée comme un seul objet.

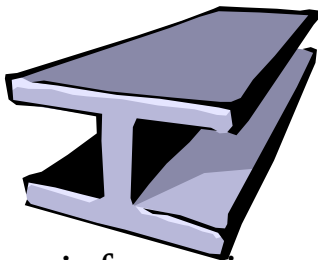
La définition de primitives de traitement et de manipulations de ces objets.

Exemple : Une Pile est caractérisée par le principe de FILO (First In First Out)

Spécification logique:

L'exploitation d'une structure de données passe par le choix du mode de représentation en mémoire et de l'écriture des primitives d'accès aux données.

Les Structures de Données

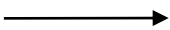


Représentation par tableau

La zone mémoire de représentation utilisée pour stocker les informations est sous forme de tableaux. Une zone est réservée pour les données proprement dites et une ou plusieurs zones pour la structure ou le lien entre les éléments



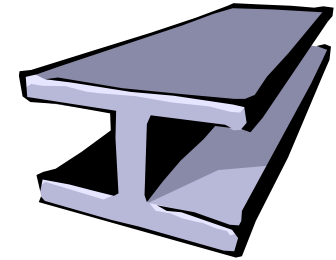
Début



C	9	2
	8	
	7	
A	6	4
	5	
B	4	9
	3	
D	2	1
E	1	0

Information/Donnée indice Indice du successeur

Les Structures de Données

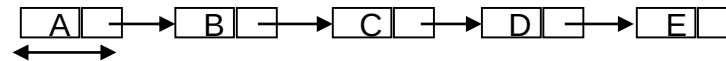


Représentation par pointeur

Il y a spécification de l'élément de base de la structure composé de deux parties. La première est réservée à l'information proprement dite, la seconde correspond au lien de chaînage entre les éléments de la structure.



Exemple :



Un élément

Les Structures de Données

Structures de données homogènes

Tableau monodimensionnel

Spécification fonctionnelle

Chaque élément de cette structure, tableau, est repérable à l'aide d'une valeur appelée indice. Cela permet un accès direct en lecture ou en écriture.

Spécification logique

Déclaration d'un tableau

```
TYPE      Nom_du_Type = TABLEAU [Nb_Element] de Telle_nature ;
```

Déclaration de variables

```
Vari_Tab  : Nom_du_Type ;      // La variable de type tableau  
Tab       : Telle_nature;      //Variable du type d'un élément du tableau  
Indice    : ENTIER ;          // Entier désignant le rang de l'élément
```

```
Vari_Tab[Indice] = Tab ;      /* Le contenu de la variable « Tab » est mis dans le tableau  
« Vari_Tab » au rang Indice. (Ce rang peut être différent selon les langages).*/
```

Remarque : nous allons voir en langage C l'utilisation du nom du tableau pour accéder à un élément.

Les Structures de Données

Structures de données homogènes

Tableau monodimensionnel

Spécification physique

La représentation de la donnée tableau dans la mémoire est faite de manière *contiguë* de taille **$P \times E$** avec **P** éléments du tableau dont chacun à une taille donnée **E** .

L'accès à un élément de rang Indice est donnée par :

$$\text{Adresse de l'élément de rang Indice} = \text{Deb} + (\text{Indice} \times E)$$

Deb est l'indice de début (!!!!!!!Ce rang peut être différent selon les langages)

Exemple (langage C)

-tableau à une seule dimension: `Tableau[Nb_elem] : entier;`



```
#define Nb_Elem 25
```

```
int Tableau_1D[Nb_Elem]; //déclaration d'un tableau a 1 dimension de 25 éléments
```

```
// Accès direct
```

```
Tableau[0] = 10 ; // affectation de la valeur 10 au premier 'élément
```

```
// Accès par pointeur
```

```
*(Tableau+11) = 3; // affectation de la valeur 3 à l'élément de rang 11 !!!!
```

Les Structures de Données

Structures de données homogènes

Tableau multidimensionnel

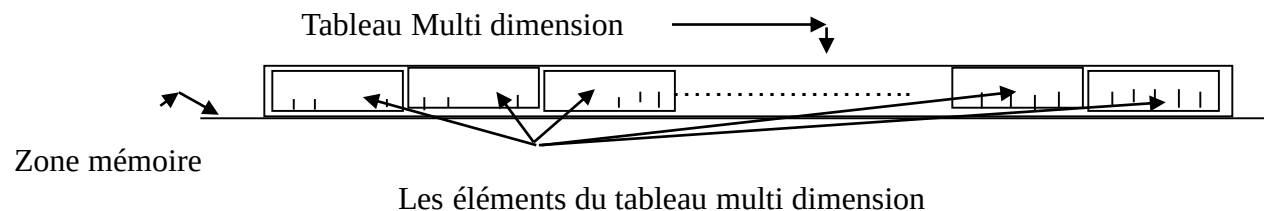
Spécification fonctionnelle

C'est une extension de la définition d'un tableau monodimensionnel. Elle permet de représenter des objets comme une matrice, une image,

Spécification logique

Un tableau multidimensionnel peut être considéré comme un tableau mono dimension de tableaux. Un élément de ce tableau est repéré grâce à plusieurs indices selon la représentation :

-Contiguë : l'ensemble des éléments du tableau multi dimension est formé d'élément côte à côte dans la zone mémoire

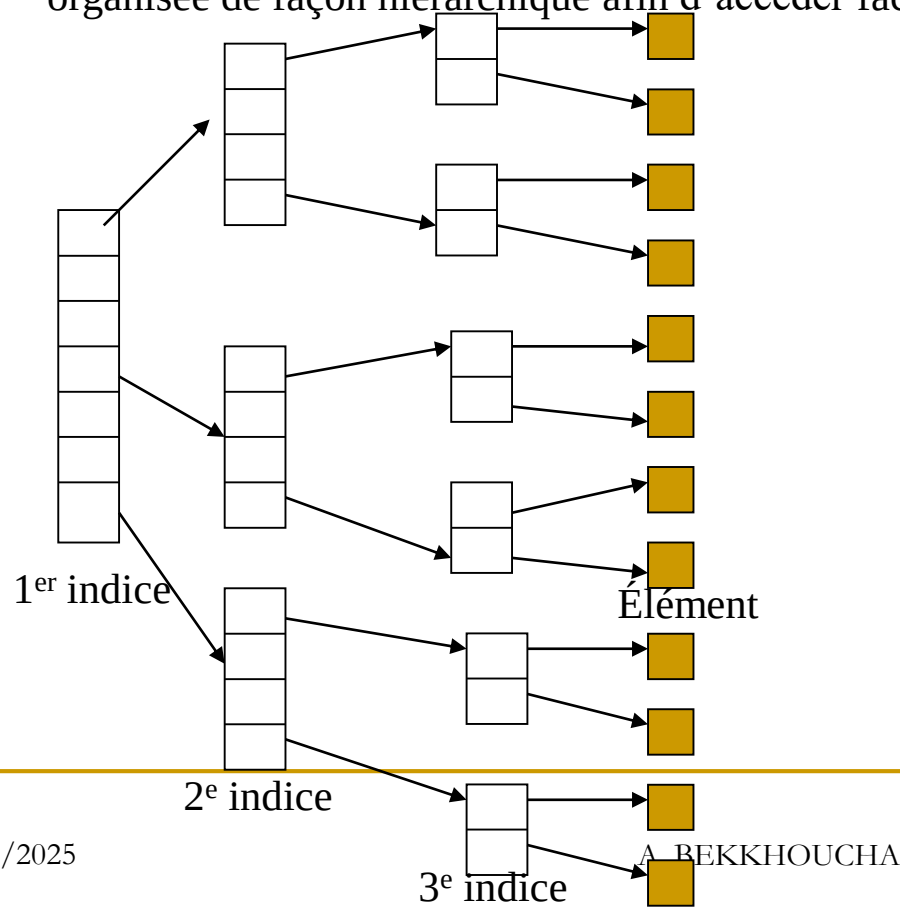


Les Structures de Données

Structures de données homogènes

Tableau multidimensionnel

-Chaînée : cela est intéressant pour les tableaux de très grande taille. L'information est organisée de façon hiérarchique afin d'accéder facilement à l'information désirée.



Les Structures de Données

Structures de données homogènes

Tableau multidimensionnel

Exemple :

- Tableau à deux dimensions : `Bi_Tableau[Max_elem][Nb_elem] : entier;`

Affectation d'une valeur à un élément du tableau

`Bi_Tableau[5][10] <-- val; /* affectation de "val" à l'élément du tableau identifié par les positions (5,10) dans la matrice Bi_Tableau */`

Accès à un élément

- accès directe par indice:

`Bi_Tableau[n° ligne] [n° colonne]`

- accès par adresse (pointeur en C)

`*(Bi_Tableau[n° ligne] +n° colonne)`

Les Structures de Données

Structures de données homogènes

Enregistrement

L'enregistrement est une structure de données hétérogène composée d'un nombre fixe d'information de type différent.

Spécification fonctionnelle

Un enregistrement est un ensemble d'informations de type différent accessible individuellement ou collectivement en lecture et écriture. Des exemples dans la vie courante sont très nombreux : identification d'un étudiant et de ses notes, représentation d'un abonné de téléphone, information relative à un assuré.....

Les Structures de Données

Structures de données homogènes

Enregistrement

Spécification logique

Grâce aux mots clés ENREGISTREMENT, STRUCT ou RECORD il y a définition d'une structure composée représentant un seul individu. La forme générale d'une déclaration d'enregistrement est la suivante :

```
TYPE Nom_de_Type = ENREGISTREMENT
    Champ1 : Type1 ;
    Champ2 : Type2 ;
    .....
    ChampN : TypeN ;
Fin
```

Cette déclaration définit une structure d'enregistrement de nom Nom_de_Type composée des champs respectifs Champ1 de type1 ChampN de TypeN.

Ces champs peuvent être de type simple ou composé, de type prédéfini ou défini par l'utilisateur. Nous pouvons avoir des tableaux dont les éléments sont structurés (enregistrement). L'accès aux différents champs de la structure passe par la désignation de l'indice de élément de ce tableau.

Les Structures de Données

Structures de données homogènes

Enregistrement

Spécification physique

Les enregistrements sont stockés dans la mémoire de façon contiguë. La taille de la zone mémoire réservée à la structure de type enregistrement est égale à celle de la somme des tailles de tous les champs la composant.

Exemple :

Soit la structure suivante :

```
structure
{ champ1 T1;
  champ2 T2;
  .....
} Enregistrement
```

Les Structures de Données

Structures de données homogènes

Enregistrement

-Tableau dans les éléments sont des enregistrements

Tableau_enreg[NB_elem] : Enregistrement.

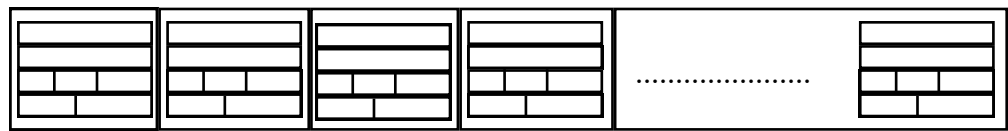


Tableau dont les éléments sont des enregistrements

Accès à un enregistrement d'un élément du tableau :

Tableau_enreg[Ind_elem].champ1 ;

Les Structures de Données

Structures de données homogènes

Recherche linéaire

Lorsqu'on a peu d'information sur les données, l'approche la plus simple consiste à explorer séquentiellement le tableau de façon à augmenter pas à pas la partie du tableau dans laquelle on sait que ne figure pas l'élément recherché. C'est ce que nous appelons la **recherche linéaire**. La recherche est interrompue sous l'une des deux conditions :

L'élément recherché a été trouvé.

Le tableau a été entièrement parcouru sans trouver l'élément.

Algorithme :

// L'élément recherché est Elem_Recher ;

Indice = 0 ;

Tantque ((indice < Max_Elem) et (Tableau[indice] ≠ Elem_Recher))

Faire Indice = Indice+1 ;

Les Structures de Données

Structures de données homogènes

Recherche dichotomique

Si nous disposons d'information sur l'organisation des données, le processus de recherche s'en trouve accéléré. La recherche parmi des données organisées est plus efficace, exemple du dictionnaire.

L'idée de base est de choisir un élément au hasard du tableau de données et de le comparer à l'élément recherché :

Si Elem_Recher est égale à l'élément du tableau choisi alors la recherche est fructueuse, nous l'avons trouvé et nous arrêtons la recherche.

Sinon, si Elem_Recher est inférieur à l'élément choisi du tableau. Cela veut dire que les éléments se trouvant dans la partie supérieure à l'élément du tableau choisi sont écartés de la recherche. La recherche se poursuit la partie inférieure à celle de l'élément du tableau choisi

Sinon, si Elem_Recher est supérieur à l'élément choisi du tableau. Cela veut dire que les éléments se trouvant dans la partie inférieure à l'élément du tableau choisi sont écartés de la recherche. La recherche se poursuit dans la partie supérieure à celle de l'élément du tableau choisi.

Les Structures de Données

Structures de données homogènes

Recherche dichotomique: algorithme

```
Gauche = 0 // Borne de gauche
Droite = Max-Elem ; // Borne de droite
Touve = Faux ; // Booléen
Indice : ENTIER ; // Une variable
Tant que ((Gauche ≤ Droite) et (Non Trouve))
    Faire
        Indice = toute valeur entre Gauche et Droite, en général (gauche+droite)/2
        Si (Tableau[Indice] = Elem_Recher) alors Trouve = Vrai ;
        Sinon
            Si (Tableau[Indice] < Elem_Recher)
                alors
                    Gauche = Indice + 1 // Déplacement de la borne de gauche
            Sinon
                Droite = Indice - 1 ; // Déplacement de la borne de droite.
        Finsi
    Finsi
Fintanque.
```

Remarque : Le choix de la valeur Indice a de l'importance. Le but est d'éliminer un maximum d'information inutile.

Les Structures de Données

Structures de données homogènes

Recherche dichotomique: algorithme amélioré

L'algorithme peut être amélioré de la façon suivante :

Gauche = 0 // Borne de gauche

Droite = Max-Elem ; // Borne de droite

Indice : ENTIER ; // Une variable

Tant que (Gauche < Droite)

 Faire

 Indice = (gauche+droite)/2 ;

 Si (Tableau[Indice] < Elem_Recher) alors

 Gauche = Indice + 1 // Déplacement de la borne de gauche

 Sinon Droite = Indice ; // Déplacement de la borne de droite.

 Finsi

Fintanque.

La condition “(Gauche < Droite)” sera obligatoirement atteinte et produira l’arrêt de l’algorithme.

Les Structures de Données

Structures de données homogènes

Manipulation des éléments d'un tableau: insertion

Soit la structure suivante

Structure

```
{  
  int Tableau_1D[Nb_Elem] ;      // déclaration du tableau  
  int Rang_Dernier_elem ;        /* Variable contenant l'emplacement du dernier  
  }                               Élément mis dans le tableau*/
```

Insertion au rang « Indice »

Exemple :

```
int ind_boucle ; // Variable indiquant la boucle.  
for (nd_boucle = Rang_Dernier_elem ;                // Début depuis la fin du tableau  
    ind_boucle >= Indice ;                          // test d'arrêt lorsqu'on arrive sur l'élément de rang indice  
    Tableau_1D[ind_boucle+1] = Tableau_1D[ind_boucle--]) ; // fin de la boucle for  
    /* La dernière instruction de la boucle « for » réalise à la fois une affectation puis une  
    décrémentation de l'indice de boucle.*/  
    Rang_Dernier_elem++ ; // augmentation du nombre d'élément dans le tableau
```

Les Structures de Données

Structures de données homogènes

Manipulation des éléments d'un tableau: suppression

Exemple :

L'élément à supprimer est de rang *indice*.

```
int ind_boucle ; // Variable indiquant la boucle.
for ( ind_boucle = Indice ;                               // Début depuis la fin du tableau
      ind_boucle < Rang_Dernier_elem;                     // test d'arrêt strict
      ind_boucle++)                                       // incrémentation de l'indice de boucle.
    Tableau_1D[ind_boucle] = Tableau_1D[ind_boucle+1]
    // La dernière instruction réalise une affectation
-- Rang_Dernier_elem ; // Diminution du nombre d'élément dans le tableau
```

Si les éléments ne sont pas côte à côte, mais aléatoirement dans le tableau il est difficile de procéder à la suppression d'un élément.

Comment indiquer les emplacements libres dans le tableau ?

Le seul moyen est de mettre une valeur particulière, ***qui n'est pas succssible d'être prise par l'un des éléments du tableau***, indiquant les emplacements libres.

~~(Personnellement je ne suis pas pour ce type de raisonnement !!!!!!!!!!!!!!!!!!!!!!!!!!!!!)~~

Les Structures de Données

Structures de données dynamiques

- Pour représenter des données dont le nombre ou la taille n'est pas connu à l'avance
- Ne pas arriver à saturation ou la place réservée est tellement surdimensionnée qu'elle n'est pas entièrement exploitée.

1-Représentation physique des structures dynamiques

Le principe repose sur le fait de réserver ou d'allouer de la place si le nombre de données arrive à augmenter et d'en libérer s'il diminue.

Les procédures de réservation et libération de la mémoire

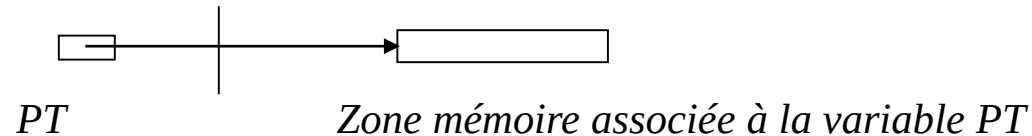
Ce sont des notions qui utilisent exclusivement les pointeurs :

a-**RESERVER** (malloc()) en langage C)

Soit un **PT** un pointeur d'une variable de type **TYPE_PT**. La fonction **RESERVER (PT)** réserve dans la zone mémoire une région pointée par **PT** utilisable par une variable de type **TYPE_PT**. La place ainsi réservée est accessible par **PT[^]**, désignant l'objet pointé le pointeur **PT**. (**PT[^]** c'est une notation algorithmique générale)

Les Structures de Données

Structures de données dynamiques



Exemple en langage C

```
int *pt_entier ;           // pointeur sur une valeur entière.  
// La variable n'est exploitable qu'après allocation de la zone mémoire.  
pt_entier = (int *) malloc(sizeof(int)) ;  
if ( ! pt_entier )  
    { printf("\n Erreur d'allocation de zone mémoire");  
      exit(1) ;  
    }  
pt_entier = 1024 ; // affectation d'une valeur à la zone mémoire
```

b-LIBERER (free en langage C)

L'opération de libération de la zone mémoire casse le lien entre la variable *PT* et la zone mémoire. Cette dernière est restituée et devient exploitable par d'autres variables. Si la libération de la zone mémoire s'effectue correctement, *PT* prend la valeur NIL (NULL en langage C)

Les Structures de Données

Structures de données dynamiques

2-Enregistrement récursifs

Un Type structuré ou un enregistrement est composé de plusieurs champs de natures différentes, simple ou enregistrement. Cet enregistrement peut être un pointeur sur lui-même. Dans ce cas nous avons un enregistrement récursif. Cela autorise d'engendrer des structures dynamiques de taille infinie.

Cette notion est exploitée dans les paragraphes ci-dessous.

Les Structures de Données

Les listes linéaires

a- Spécification fonctionnelle

Une liste linéaire est un ensemble ordonné et extensible d'éléments de même type auxquels on accède séquentiellement, et où l'on peut ajouter ou retrancher un élément en n'importe quelle position.

b- Spécification logique

Une liste est décomposée en deux parties: Tête de liste et la suite. Une propriété importante des listes est que leurs éléments peuvent être rangés suivant l'ordre de leur apparition. On dit que **a_i précède a_{i+1}** , pour $i=1\dots N$, **a_i** élément de la suite et N désigne la longueur de la liste.

Remarque : comme l'accès aux éléments de la liste se fait par la tête, si celle-ci est perdue, le reste des données lui aussi est perdu.

Les Structures de Données

Les listes linéaires

Pour manipuler l'ensemble des données représentées par des listes, nous définissons un ensemble de primitives sur ces listes. Pour ce faire nous nous donnons:

- L : liste d'objet de type "Type_Elem".
- Elem : un élément de type "Type_Elem".
- p : la position d'un élément donné dans la liste.

Les primitives:

- **INSERER_LISTE(Elem,p,L)** : permet d'insérer l'élément "Elem" à la position "p" dans la liste "L".
- **LOCALISER_LISTE(Elem,L)** : cette fonction retourne la position de "Elem" dans la liste "L".
- **ACCEDER_LISTE(p,L)** : cette fonction retourne l'élément à la position "p" de la liste "L", sans le retirer de la liste. L'opération est impossible si la position "p" n'est pas dans "L".
- **SUPPRIMER_LISTE(p,L)** : cette fonction supprime l'élément à la position p de la liste L.
- **SUIVANT_LISTE(p,L), PRECEDENT_LISTE(p,L)** : ces fonctions retournent respectivement la position suivant ou précédant la position p dans la liste L.
- **TAILLE_LISTE(L)** : elle retourne la taille de la liste, 0 si la liste est vide.
- **RAZ_LISTE(L)** : supprime, proprement, tous les éléments de la liste L. L devient une liste vide.
- **LISTER_LISTE(L)** : affiche les éléments de L dans l'ordre de leur position.

Les Structures de Données

Les listes linéaires

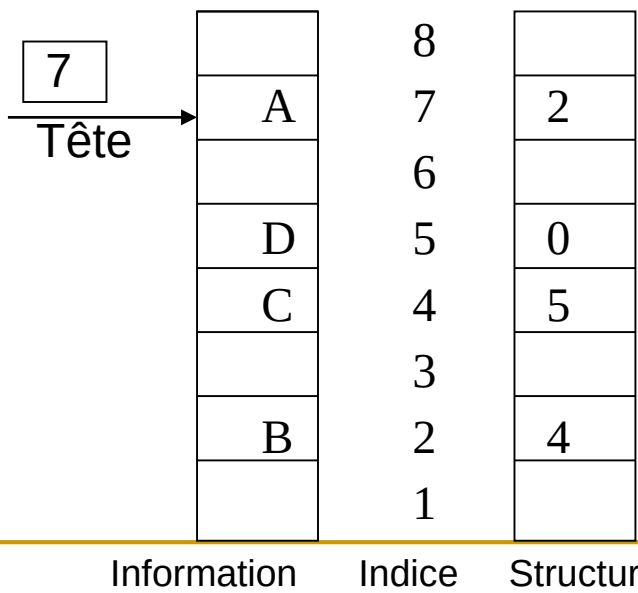
c- Spécification physique

La représentation des listes en mémoire est réalisable de deux manières différentes :

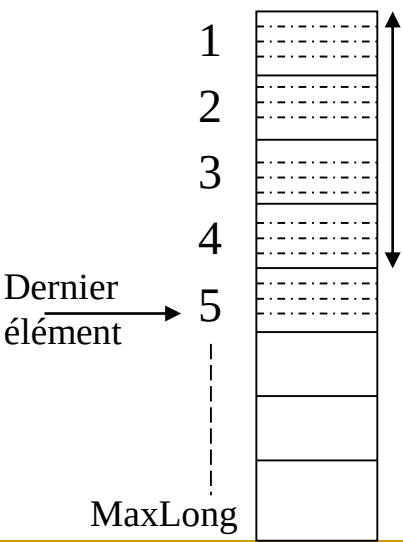
1-Représentation par tableau :

Elle est réalisée de deux façons différentes :

A l'ai de deux tableaux et d'une variable représentant la tête de la liste.



A L'aide d'un seul tableau



Les Structures de Données

Les listes linéaires

Mise en œuvre à l'aide des pointeurs

Un maillon de la chaîne est formé d'une partie information et d'un pointeur sur un autre maillon de la chaîne. Tous les maillon de la chaîne sont de même type. Nous parlerons aussi de cellule de la liste chaînée

Définition de type structure de maillon

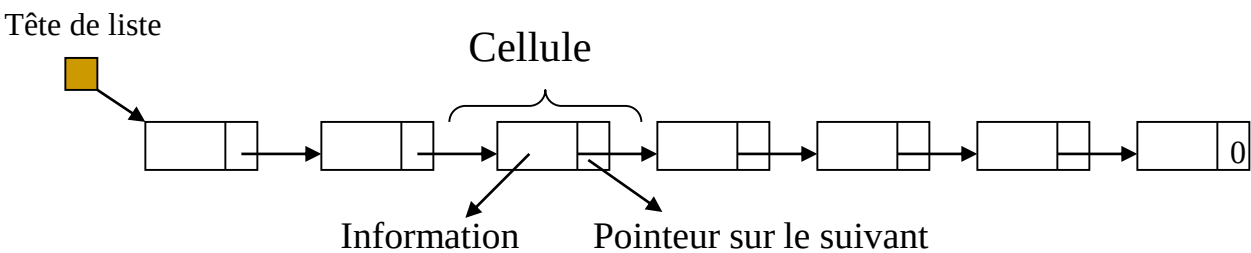
```
{  
    sous structure information;  
    pointeur sur le suivant [et sur le précédent]  
} nouveau type Cellule
```

Les Structures de Données

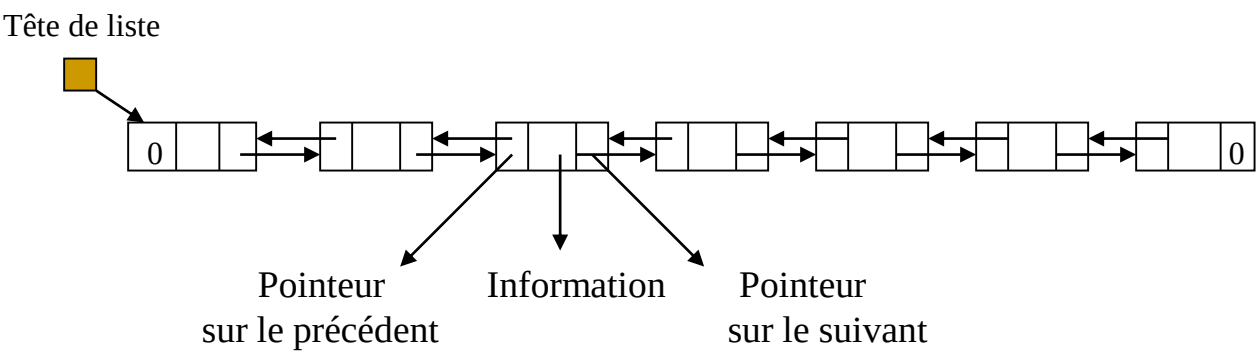
Les listes linéaires

Mise en œuvre à l'aide des pointeurs

Liste simplement chaînée



Liste doublement chaînée



Les Structures de Données

Les listes linéaires

Manipulation des listes chaînées: insertion d'un élément dans la liste

L'insertion d'un élément dans une liste passe par trois phases:

- 1) Allocation la place mémoire pour cet élément et mise en place de l'information
- 2) Raccorder au nouvel élément son suivant
- 3) Raccorder ce nouvel élément à la liste

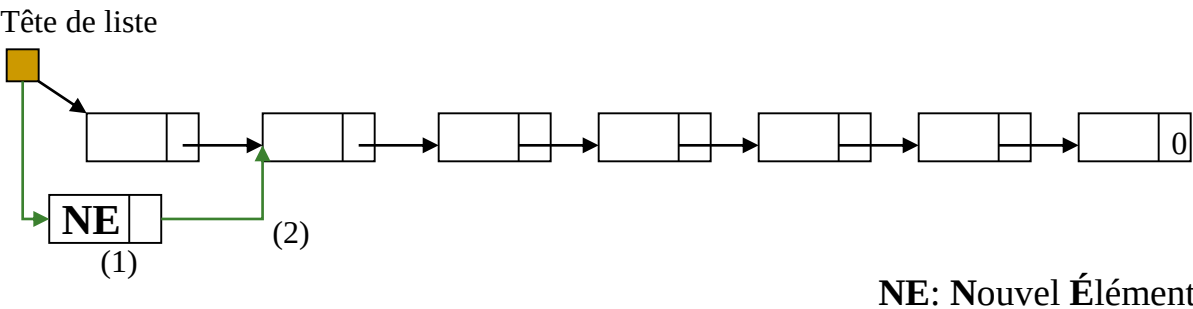
Attention : L'ordre des instructions à beaucoup d'importance

Les Structures de Données

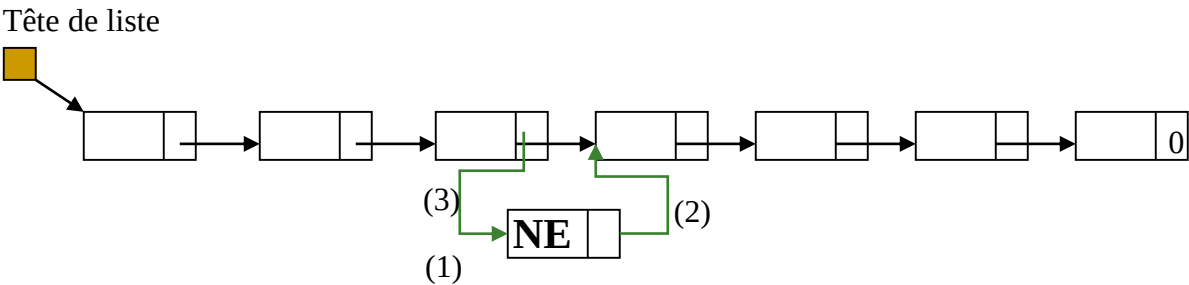
Les listes linéaires

Manipulation des listes chaînées: insertion d'un élément dans la liste

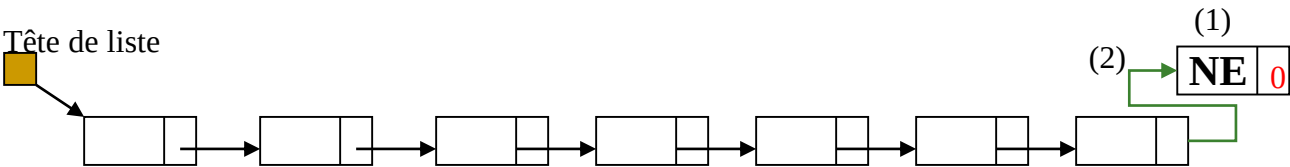
Insertion en tête de liste



Insertion au milieu de la liste



Insertion en queue de liste



Les Structures de Données

Les listes linéaires

Manipulation des listes chaînées: Algorithme d'insertion d'un élément dans la liste

```
Cellule *INSERER_LISTE( elem, p, liste)
Cellule *elem;
entier p;
Cellule *liste;
debut
  Cellule *New_elem , *Pt_courant;
  si (TAILLE_LISTE(L) < (p-1) ) alors message d'erreur «liste trop courte»
  sinon
    debut    /* réserver de la place mémoire pour la nouvelle cellule */
      New_elem = Allouer(Cellule);
      New_elem^elem = elem;
      Pt_courant = liste;
      si (p = 1) alors
        debut
          New_elem^suivant = liste;
          retourne(New_elem);
        fin
    fin
  fin
```

/ insertion en tête de liste*/*

Les Structures de Données

Les listes linéaires

```

sinon
si (TAILLE_LISTE(L) = p-1) alors /* insertion en queue de liste*/
    debut
        tant que (pt_courant^suivant^suivant ≠ NULL)      ???????
            pt_courant = pt_courant^suivant;
            pt_courant^suivant = New_elem;
            New_elem^suivant = NULL;
        fin
    sinon /* détermination de la position */
        debut
            position = 1;
            tantque (position < p) faire
                debut
                    pt_courant = pt_courant^suivant;
                    position++;
                fin
            New_elem^suivant = pt_courant^suivant;
            pt_courant^suiavnt = New_elem;
        fin
    retourne(liste);
fin
fin

```

Les Structures de Données

Les files d'attentes



a- Spécification Fonctionnelle

Dans la file les éléments sont insérés en queue et supprimés en tête (ou inversement). Le nom vient des files d'attente à un guichet, où le premier arrivé est le premier partie «First In, First Out: FIFO».

b- Spécification logique

La file d'attente peut être décomposée en deux partie : la tête (celui qui va être retiré) et le reste des éléments. Un ensemble de fonctions de manipulation des listes d'attentes est spécifié en fonction de la définition de la structure de données. La caractérisation de la liste est faite soit par sa tête seule soit à l'aide de la tête et de la queue.

Remarque :

Puisque la file est active en tête pour défiler et en queue pour enfiler il est judicieux de garder deux indicateurs: l'un de tête, l'autre de queue afin de faciliter les opérations sur la file.

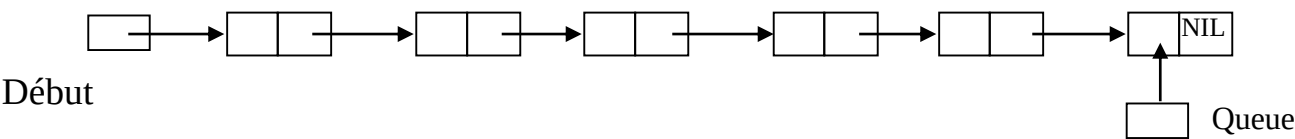
Les Structures de Données

Les files d'attentes

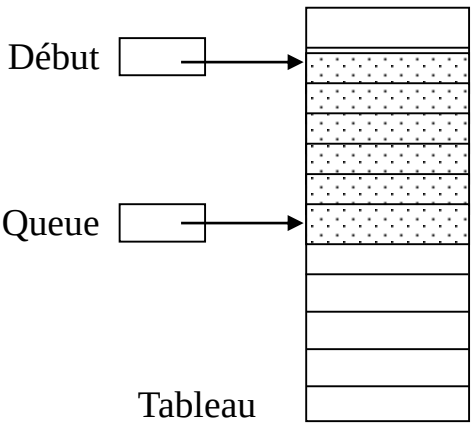
c- Spécification physique

Dans ce cas, nous gardons des indicateurs sur la tête et sur la queue comme les indique les schémas suivants :

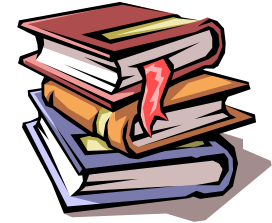
Par pointeurs



Par tableaux : représentation contiguë



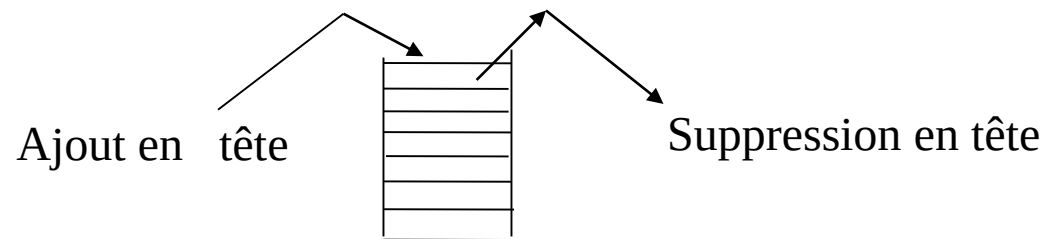
Les Structures de Données



Les Piles

a- Spécification fonctionnelle

Elle correspond à la notion de la pile d'assiettes, la pile de dossiers,..... La caractéristique essentielle d'une pile est que l'on retire toujours d'abord le dernier élément déposé, l'ordre de sortie étant donc l'inverse de l'ordre d'entrée. Le terme anglo-saxon utilisé est LIFO (Last In First Out)



Les Structures de Données

Les Piles

b- Spécification logique

Une pile est définie par un sommet et un corps. Tout élément de la pile est à un moment donné tête de pile. Une pile peut être définie comme une structure de données Dynamique homogène à un seul point d'accès, les données sont ajoutées ou retranchées par l'intermédiaire d'une tête d'accès appelée sommet de pile. Les opérations caractéristiques d'une pile sont:

- ***EMPILER(Elem,Pile)*** : *ajoute un élément au sommet de la pile.*
- ***DEPILER(Elem,Pile)*** : *retire l'élément se trouvant au sommet de la pile.*
- ***RAZ_PILE(Pile)*** : *vide la pile*
- ***SOMMET_PILE(Pile)*** : *retourne le sommet sans dépiler cet élément de la pile.*
- ***VIDE_PILE(Pile)*** : *retourne Vrai si pile est vide faux sinon.*

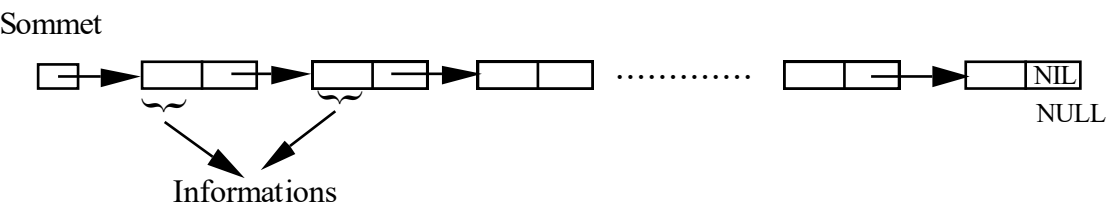
Les Structures de Données

Les Piles

c- Spécification physique

Une pile est une suite avec un point d'accès unique qui peut être représenté par pointeur ou tableau.

*Représentation chaînée: elle est résumée par le schéma suivant:



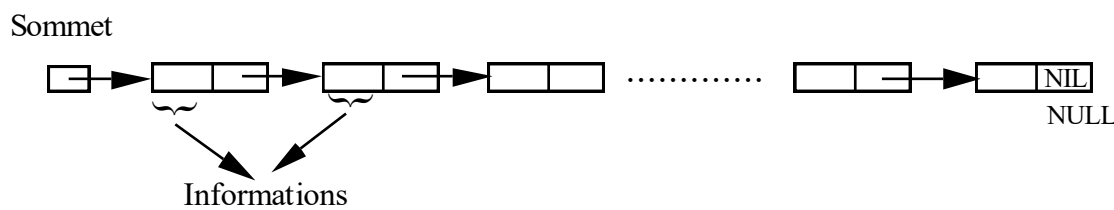
Les Structures de Données

Les Piles

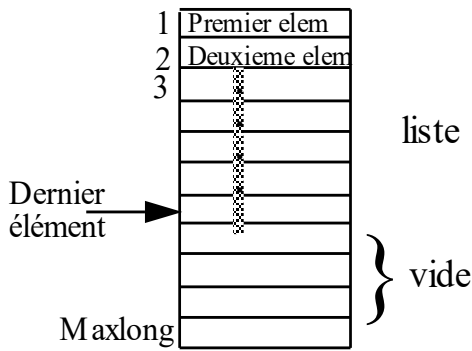
c- *Spécification physique*

Une pile est une suite avec un point d'accès unique qui peut être représenté par pointeur ou tableau.

*Représentation chaînée:



*Représentation contiguë (tableau)



Les Structures de Données

Les arbres

Notions sur les arbres:

Spécification fonctionnelle

Un arbre est une collection d'informations homogènes organisée en niveaux, où chaque information d'un niveau donné peut être reliée à plusieurs informations du niveau inférieur par des branches ne formant pas des boucles, et représentant soit un lien de nature particulière (sémantique, hiérarchique, relation d'ordre, etc...) soit une transition ou une action possible.

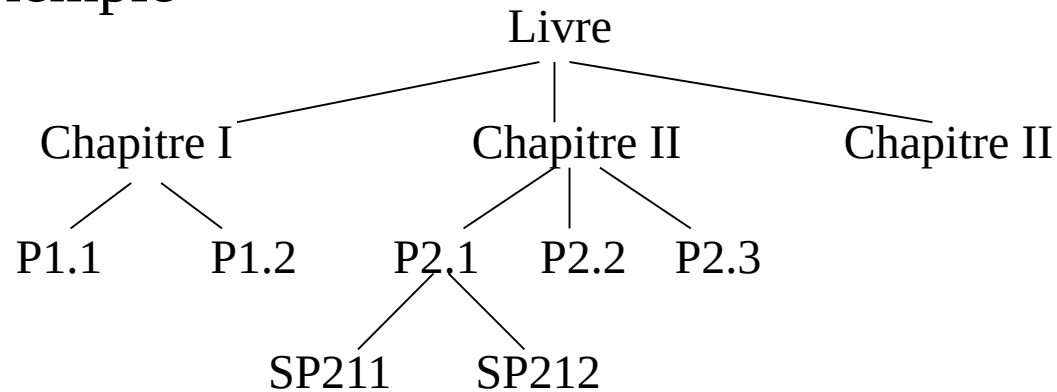
Exploitation

Méthodes **de tris, de gestion d'information**,... , en **compilation** (analyse syntaxique et lexicographique) ou bien dans le domaine de **l'intelligence artificielle**.

Les Structures de Données

Les arbres

Exemple



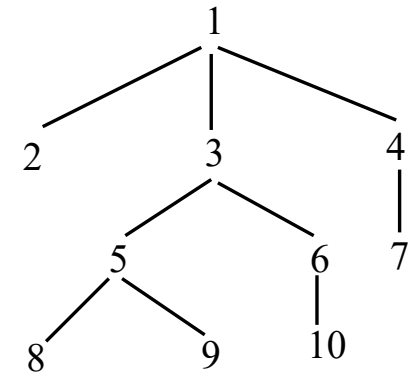
Racine : un élément particulier de cet arbre est la racine qui a lui seul forme un arbre.

Nœud : sont des éléments de l'arbre qui a leur tours forment des sous arbres.

Les Structures de Données

Les arbres

Caractéristiques d'un arbre



La longueur de l'arbre est égale au nombre de nœud moins un pour la branche la plus longue.

Un nœud unique a une longueur égale à 0.

Le passage d'un nœud à un autre s'effectue par un chemin.

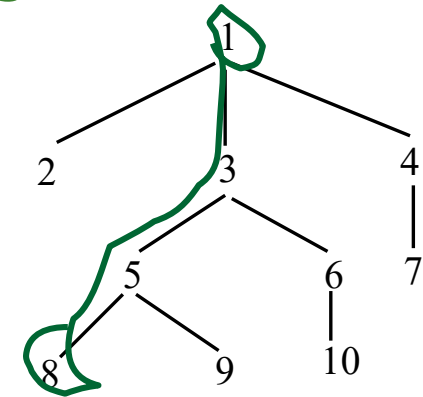
Si un chemin existe entre deux nœuds a et b alors on dit que a est un **ascendant** ou un **ancêtre** de b. Un ascendant ou descendant d'un nœud autre que lui même s'appelle ascendant ou descendant propre. Dans un arbre seul la racine n'a pas d'ascendant propre.

Une racine ou un nœud sans descendants propre est une ***feuille***.

Les Structures de Données

Les arbres

Ordre sur les nœuds d'un arbre



Les fils d'un nœud sont habituellement ordonnés de gauche à droite. L'ordre «gauche-droite» permettant la comparaison d'enfants d'un même père peut être prolongé pour comparer deux nœuds qui ne sont pas liés par la relation ascendant-descendant.

Pour trouver les nœuds situés à droite ou à gauche d'un nœud, il faut tracer le chemin reliant la racine à ce dernier toutes les «branches» de l'arbre partant sur la gauche de ce chemin aboutissent aux nœuds situés à gauche. Tous les nœuds issus de branches sur la droite du chemin sont à droite.

Le nœud 8 est à droite du nœud 2 et à gauche des nœuds 9, 6, 4, 10 et 7. Il n'est ni à gauche ni à droite des ascendants 1, 3, 5.

Les Structures de Données

Les arbres

Spécification logique des arbres

La définition d'un arbre est récursive, se réduit soit à un arbre vide soit formé d'une racine et de sous-arbres comme fils. Sur les arbres quelconques, les algorithmes usuels sont les parcours, sur les arbres ordonnés en plus des primitives standards de recherche il y a les primitives d'ajout et de retrait d'élément avec conservation de la propriété d'ordre qui qualifie l'arbre.

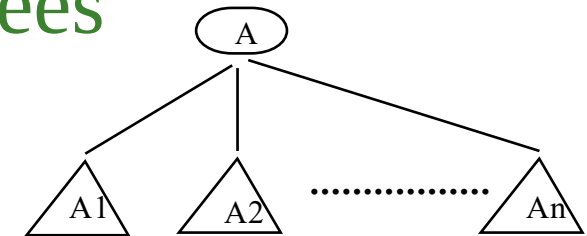
Parcours d'arbres

Il existe plusieurs moyens de parcourir les nœuds d'un arbre. Les trois parcours les plus importants sont les parcours **préfixé** (pré ordre), **postfixé** (post ordre) et **infixé** (ordre); ces parcours sont définis récursivement de la manière suivante:

- Si un arbre A est vide, la liste vide constitue le parcours préfixé, infixé et postfixé de A.
- Si A consiste en un seul nœud, la liste composée de ce nœud constitue le parcours préfixé, infixé et postfixé de A.

Les Structures de Données

Les arbres



Spécification logique: parcours d'arbres

Sinon, supposons que “A” est un arbre de racine n à k sous-arbres A1, A2,Ak comme celui de la figure ci dessous.

1- pour effectuer le parcours «**préfixé**» des nœuds de “A”, on commence par la racine n puis on entreprends le parcours préfixé des nœuds de A1, de ceux de A2 et ainsi de suite jusqu’aux nœuds de Ak.

2- pour effectuer le parcours «**infixé**» des nœuds de “A”, on commence par le parcours infixé du nœud de A1, puis on passe par la racine n, puis on effectue le parcours infixé des nœuds de A2.....Ak.

3- pour effectuer le parcours «**postfixé**» des nœuds de “A”, on effectue le parcours postfixé des nœuds de A1 puis de A2 et ainsi de suite jusqu’a Ak, et l’on termine par la racine.

Les Structures de Données

Les arbres

Parcours d'arbres: les algorithmes

procédure **PREFIXÉ**(n : nœud)

debut **lister** n

pour chaque fils « f » de n , s'il y en a depuis le plus gauche jusqu'au plus à droite

faire

PREFIXÉ(f)

fin

procédure **INFIXE**(n : nœud)

debut si n est une feuille alors lister n

sinon debut

INFIXE(fils le plus à gauche de n)

lister n

pour chaque fils « f » de n , à l'exception du plus à gauche
depuis le plus gauche jusqu'au plus à droite

faire

INFIXE(f)

fin

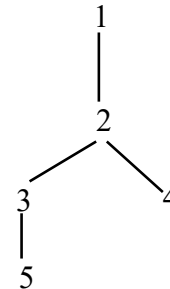
fin

Les Structures de Données

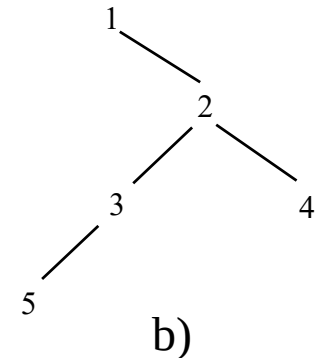
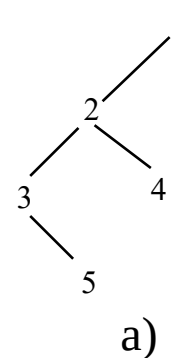
Les arbres

Arbres Binaires

Soit l'arbre «ordinaire» suivant:



Si cet arbre est ordonné et orienté, c.a.d les fils sont ordonnés de gauche à droite nous obtenons les configurations suivantes:



Les parcours des arbres binaires en ordre préfixé ou postfixé sont analogues à ceux des arbres ordinaires. Le parcours infixé des noeuds des arbres ci-dessus nous donne:

L'arbre a: 35241

L'arbre b: 15324

Les Structures de Données

Les arbres

Mise en œuvre des arbres

Représentation des arbres binaires

Nous supposons que les nœuds sont numérotés de 1 à n et que l’on utilise un tableau d’enregistrement.
Déclaration:

```
Nœud = Tableau[1..MaxNœuds] d’enregistrements
    fils_gche: entier;
    fils_dt   : entier;
fin
```

Donc
Tableau[i].fils_gche représente le fils gauche du nœud i.
Tableau[i].fils_dt représente le fils droit du nœud i.

Nœud	fils gauche	fils droit
1	2	0
2	3	4
3	0	5
4	0	0
5	0	0

Afin de pouvoir remonter un arbre, l’information relative au père peut être introduite par l’ajout d’un troisième champ dans la structure de données.

Les Structures de Données

Les arbres

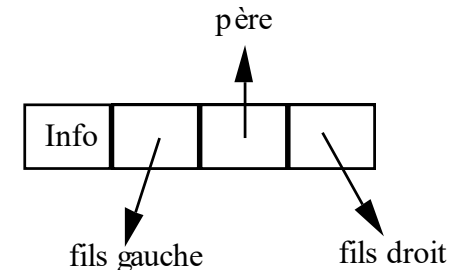
Mise en œuvre des arbres

Mise en œuvre des arbres binaires par pointeurs.

Pour cela il faut définir une structure avec éventuellement trois champs. Chaque champ pointe sur un nœud.

```
structure nœud
{
  Type          Info;
  structure nœud ^fils_gche;
  structure nœud ^fils_dt;
  structure nœud ^père;
}
```

représentation



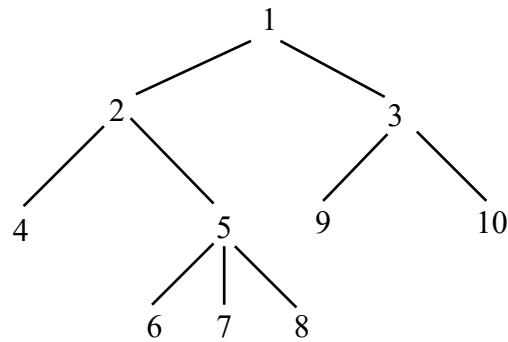
Les Structures de Données

Les arbres

Mise en œuvre des arbres

Mise en œuvre par Tableau

Soit l'arbre suivant:



Donne le tableau suivant:

Tableau

1	2	3	4	5	6	7	8	9	10	Fils
0	1	1	2	2	5	5	5	3	3	Père

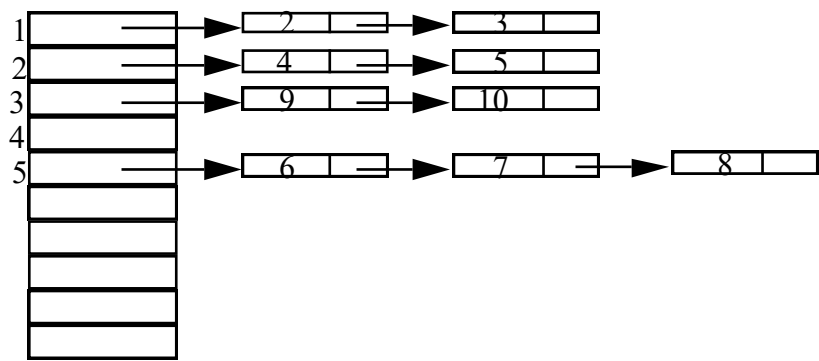
Pour chaque élément du tableau $T[i]=j$ si le nœud j est le père du nœud i et $T[i] = 0$ si le nœud est racine.

Les Structures de Données

Les arbres

Mise en œuvre des arbres

Représentation par liste des fils.



```
fonction Premier_fils(n:nœud, A: Arbre): nœud
debut
    L = tête de liste
    si la liste l est vide alors retourne ∅ /*n est feuille*/
    sinon Acceder au premier élément de la liste C.
fin
```

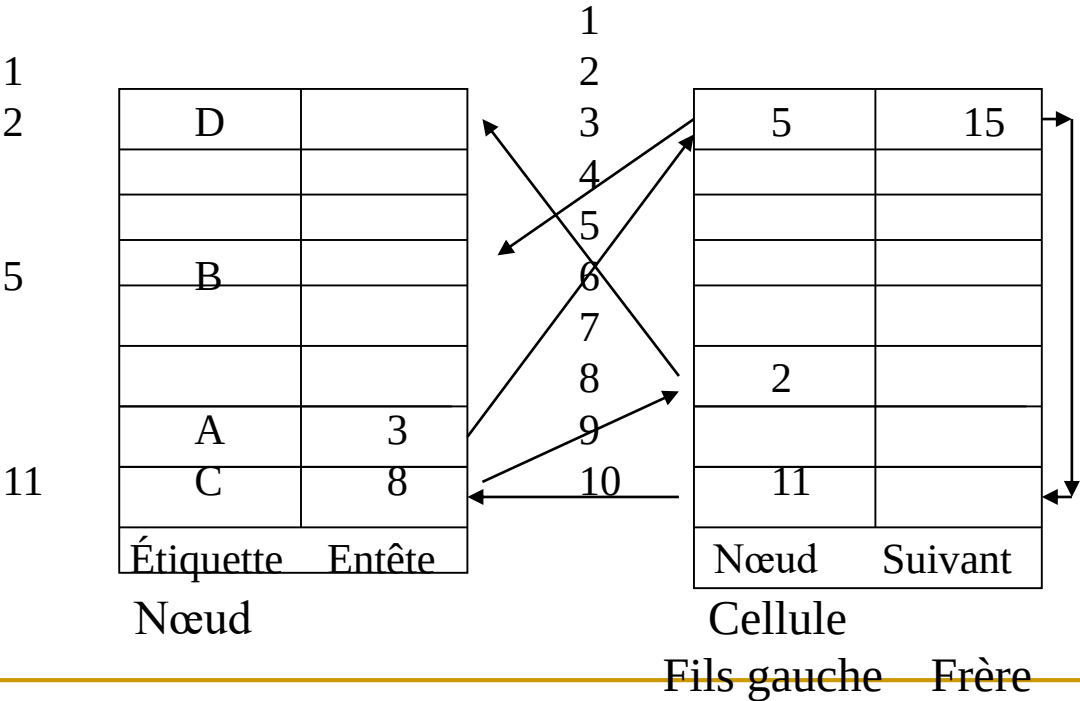
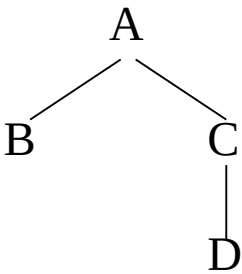
Les Structures de Données

Les arbres

Mise en œuvre des arbres

Représentation libre

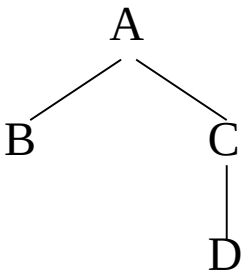
Soit l'arbre suivant :



Les Structures de Données

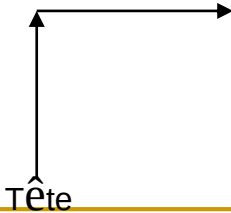
Les arbres

Mise en œuvre des arbres



Ou bien

	Fils le	plus à gauche	étiquette	frère de droite
1				
2			D	
3				
4				
5			B	9
6				
7				
8		5	A	
9		2	C	
10				



Les Structures de Données

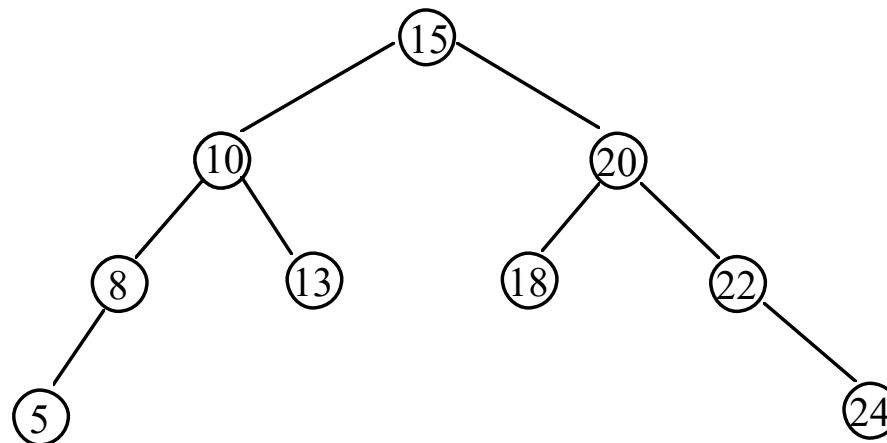
Les arbres

Arbres ordonnés

Arbre binaire ordonné horizontalement

Un arbre binaire est ordonné horizontalement (gche-droite) si la clé de tout nœud non feuille est supérieure à toutes celles de son sous arbre gauche et inférieur à toutes celles de son sous arbre droit.

Exemple:



Les Structures de Données

Les arbres

Arbres ordonnés

Recherche d'un élément

Comme la recherche est effectuée sur une partie de l'arbre, le processus se trouve accéléré. La recherche sur un arbre horizontal est dichotomique. La fonction "Action" désigne l'opération à faire sur un élément de l'arbre.

1re méthode (itérative)

X : élément à rechercher

A : Arbre binaire

Trouvé : booléen;

Recherche(X, Action, A, Trouvé)

début Trouvé <-- Faux;

tant que ((non trouvé) et (! vide_arbre(A)))

début **Si** X = A.racine **alors**

début Trouvé <-- Vrai; Action(A.racine) **fin**

Sinon

Si X < A.racine **alors**

A<--Sous_arbre gauche;

Sinon

A<-- Sous_arbre droit;

fin /* fin du tant que */

fin

Les Structures de Données

Les arbres

Arbres ordonnés

Recherche d'un élément

2e méthode (récursive)

X : élément à rechercher

A : Arbre binaire

Trouvé : booléen;

Recherche(X, Action, A, Trouvé)

début

Si Vide_Arbre(A) **alors** Trouvé <-- Faux;

Sinon

Si X= A.racine **alors**

début Trouvé <-- Vrai; Action (A.racine); **fin**

Sinon

Si X < A.racine **alors**

Recherche(X, Sous-arbre gauche, Trouvé);

sinon Recherche(X, Sous-arbre droit, Trouvé);

fin

Les Structures de Données

Les arbres

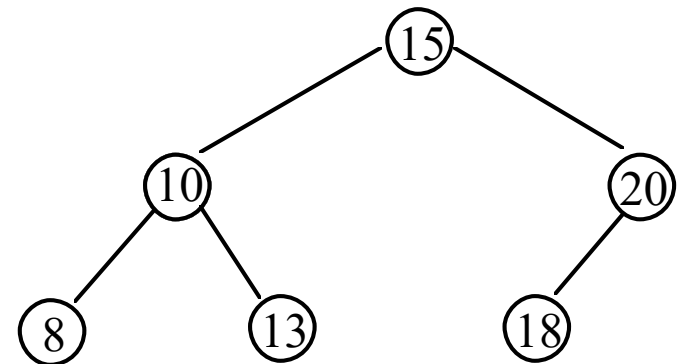
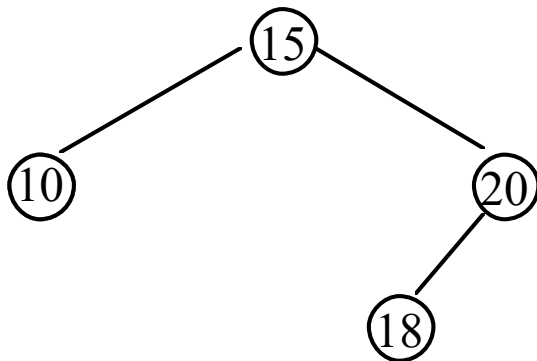
Arbres ordonnés

Insertion d'un élément

La recherche de la place d'insertion est faite récursivement, si celui-ci n'est pas déjà dans l'arbre il est mis obligatoirement en feuille.

Exemple:

Insertion de 8 et 13



Les Structures de Données

Les arbres

Arbres ordonnés

Insertion d'un élément : Algorithmes

Algorithme d'insertion

X : élément à insérer

A : Arbre binaire;

Déjà_la : indicateur booléen;

Insertion(X,A,Déjà_la)

Début

Déjà_la <-- faux;

Si Vide_arbre(A) **alors** A.racine <-- X

Sinon

Si X < A.racine **alors** Insertion(X, A.sag, Déjà_la);

Sinon

Si X > A.Racine **alors** Insertion(X, A.sad, Déjà_la);

Sinon Déjà_la <-- Vrai;

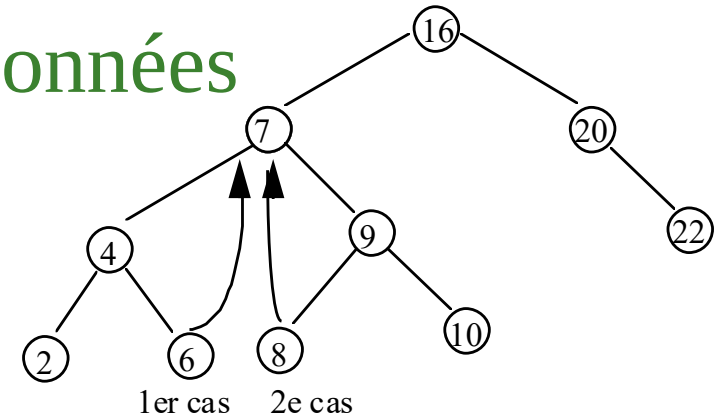
Fin

Les Structures de Données

Les arbres

Arbres ordonnés

Suppression d'un élément :



Le principe de la suppression d'un élément dans un arbre binaire ordonné horizontal est le suivant:

- si l'élément est une feuille, alors on le supprime simplement.
- l'élément n'a qu'un descendant, alors on le remplace par ce descendant.
- si l'élément à deux descendants, on le remplace aux choix soit par l'élément le plus à droite du SAG soit par l'élément le plus à gauche du S.A.D afin de conserver la propriété de l'arbre horizontal.

Exemple:

Les Structures de Données

Les arbres

Arbres ordonnés

Suppression d'un élément : Algorithme

Suppression(X,A,Trouvé)

Début

Si Vide_arbre(A) **alors** Trouvé <-- Faux;

Sinon

Si X < A.racine **alors**

Suppression (X, A.sag, Trouvé)

Sinon

Si X > A.racine **alors**

Suppression (X, A.sad, Trouvé)

Sinon

début

Trouvé <-- Vrai;

Si Vide_arbre(A.sad) **alors** A <-- A.sag

Sinon

Si Vide_arbre(A.sag) **alors** A <-- A.sad

Sinon Remplacer(A, A.sag);

fin

Fin

Les Structures de Données

Les arbres

Arbres ordonnés

Arbres binaires ordonnés verticalement :

Un arbre binaire est ordonné verticalement, si la clé de tout nœud non feuille est inférieur (respectivement supérieur) à celle de ses fils (et donc par récurrence à celle de tous ses descendants).

(Ordre partiel)

Remplacer(R, Q, I)

début

si R.dt ≠ NULL alors

début

Remplacer (R.dt, Q, I) ;

fin

Traitement après retrait dans SAD

sinon

début

Q.cle ← R.cle

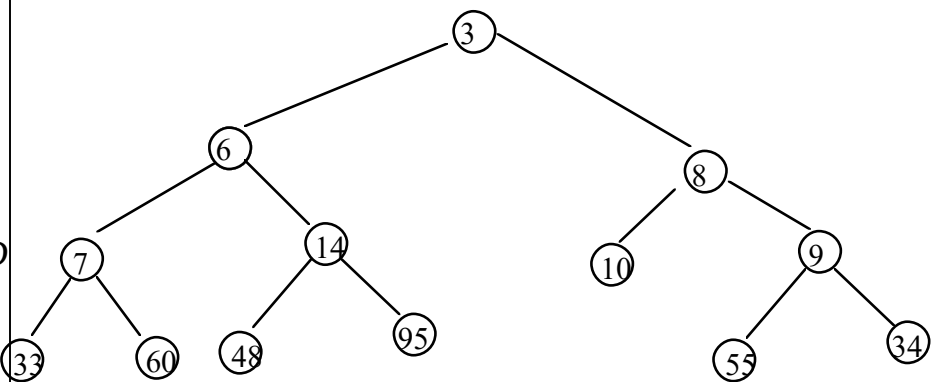
Q ← R ;

R ← R.gche ;

fin

fin

28/09/2025



Les Structures de Données

Les arbres

Arbres ordonnés

```
ARBRE *Descente(ARBRE *Arb)    // Descente à
gauche
{if (Arb->fils_dt != NULL) Descente (Arb->fils_dt);
else Arb = Arb->fils_gche;
return((ARBRE *)Arb);
}
```

// fonction qui supprime la valeur "val" de l'arbre "Arb"

```
ARBRE *Supprimer(int val, ARBRE *Arb)
{
  if (Arb== NULL) printf("\n la valeur n'est pas dans l'arbre");
  else                // Recherche dans le sous arbre gauche
    if (val < Arb->inf) Supprimer(val, Arb->fils_gche)
    else                // Recherche dans le sous arbre droit
      if (val > Arb->inf) Supprimer(val, Arb->fils_dt)
      else                // Traitement des cas restants
        if (Arb->fils_dt == NULL)
          Arb = Arb->fis_gche; //un seul descendant gauche
        else if (Arb->fils_gche == NULL)
          Arb = Arb->fis_dt;    //un seul descendant droit
        else Arb = Descente(Arb->fis_gche); // deux descendants
}
```

Les Structures de Données

Les arbres

Arbres parfaitement équilibrés

Construction d'un arbre parfaitement équilibré

La règle de distribution équitable pour un nombre de nœuds n peut se formuler récursivement:

- 1- prendre le premier nœud pour racine
- 2- générer le sous arbre de gauche avec $ng = n \text{ DIV } 2$ nœuds dans cette direction

- 3- générer le sous arbre de droite avec $nd = n - ng - 1$ nœud dans cette direction

Cette règle est exprimée sous forme d'une procédure récursive dans le programme suivant. Il lit les entrées et construit un arbre parfaitement équilibré. Un arbre est dit **parfaitement équilibré** si, pour chaque nœud, le nombre de nœud des ses sous-arbres de droite et de gauche diffèrent **d'au plus un**.

Un arbre est **équilibré** si et seulement si, pour chaque noeud, les hauteurs de ses deux sous arbres diffèrent d'au plus 1. Ces arbres sont appelés arbres **AVL** (Adelson-Velskii et Landis).

Les Structures de Données

Les arbres

Arbres parfaitement équilibrés: Programme

```
typedef struct ARB
{
    INFO info;
    Struct ARB *fils_gche, ARB *fils_dt;
} ARBRE

ARBRE *Arbre_equilibre( int n) // Fonction de construction d'arbre parfaitement équilibré
{
    // Déclaration des variables
    ARBRE *Arbre;
    int ng, nd, valeur;
    if (!n) Arbre = NULL;    //Test d'arrêt de la récursivité
    else
    {
        ng = (int)(n/2);          // Nombre de nœuds dans le sous arbre de gauche
        nd = n-ng-1;             // Nombre de nœuds dans le sous arbre de droit
        /* Construction de l'arbre parfaitement équilibré*/
        printf("\n Entrer la valeur à mettre dans votre arbre");          scanf("%d", &valeur);
        Arbre = (ARBRE *)malloc(sizeof(ARBRE));          //vérifier que l'allocation c'est bien déroulée
        Arbre->info = valeur;
        Arbre->fils_gche = Arbre_equilibre( ng); // Création du sous arbre gauche
        Arbre->fils_dt = Arbre_equilibre( nd); // Création du sous arbre droit
    }
    return (ARBRE *) Arbre;
}
```

Les Structures de Données

Les arbres

Arbres parfaitement équilibrés

Insertion dans un arbre équilibré

Lors de l'insertion d'un élément dans un arbre équilibré de racine R et de sous arbres gauche G et droit D de hauteurs respectivement hG et hD , trois cas peuvent se produire. Supposons que le nouveau nœud est inséré dans G et provoque un accroissement de hauteur 1:

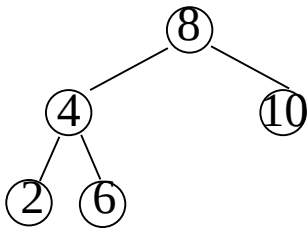
- 1 si $hG = hD$: G et D auront des hauteurs inégales, mais le critère d'équilibrage n'est pas violé
- 2 si $hG < hD$: G et D auront des hauteurs identiques et l'équilibre aura été amélioré.
- 2 si $hG > hD$: le critère d'équilibrage est violé et l'arbre doit être restructuré.

Les Structures de Données

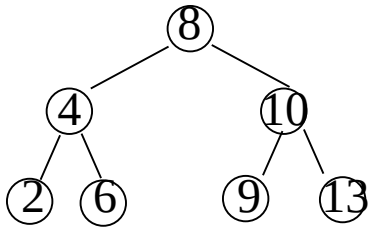
Les arbres

Arbres parfaitement équilibrés

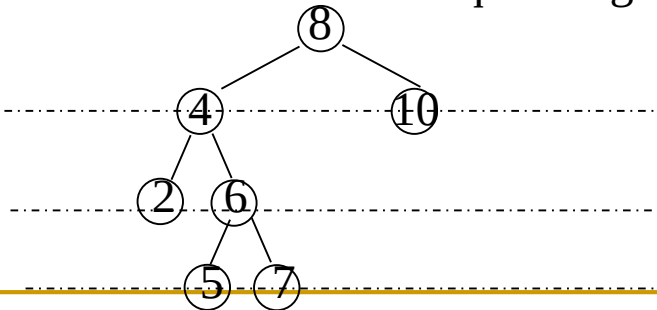
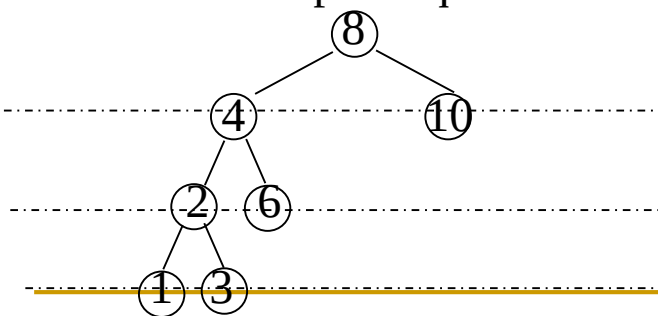
Soit l'arbre ordonné et équilibré contenant les éléments suivants:



Lors de l'insertion de 9 et 13 l'équilibre n'est pas perturbé



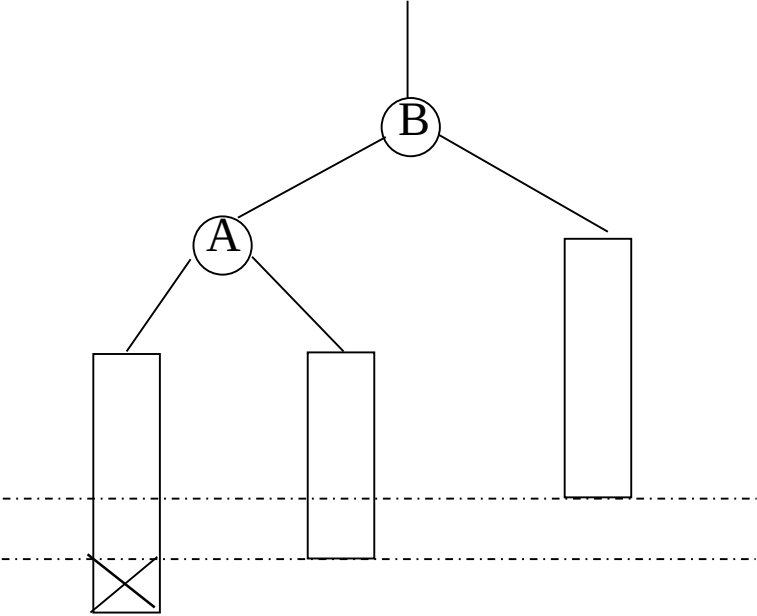
Par contre l'insertion des éléments 1 ou 3 (cas 1) (respectivement (ou 7 (cas 2)) dans l'arbre provoque un déséquilibre de l'arbre nécessite un rééquilibrage



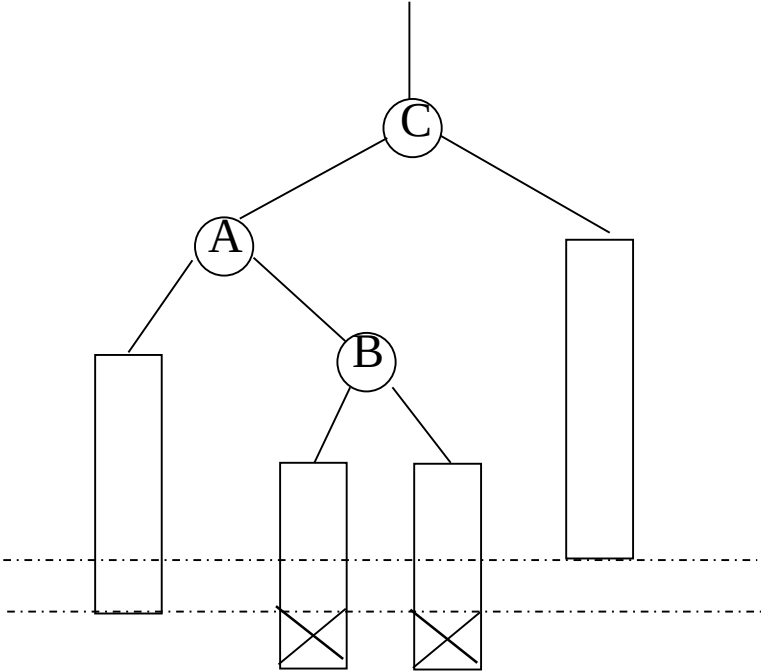
Les Structures de Données

Les arbres

Arbres parfaitement équilibrés



Cas 1



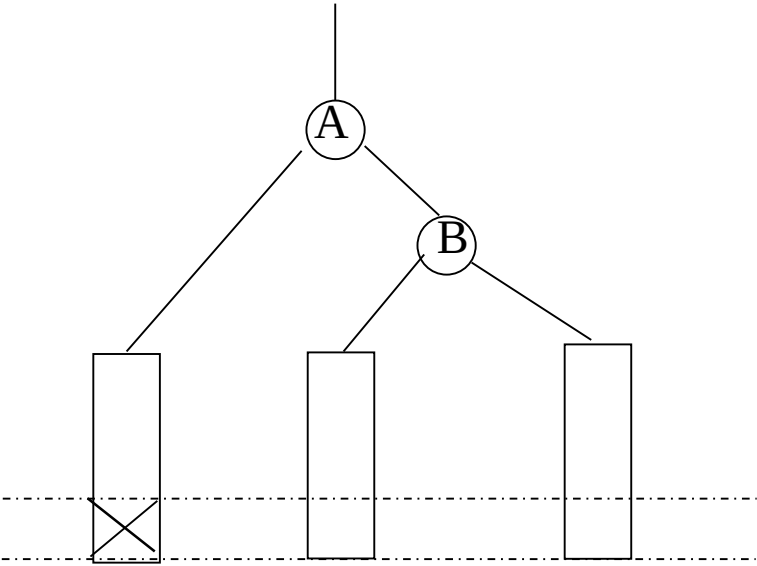
Cas 2

Un déséquilibre apparaît après insertion

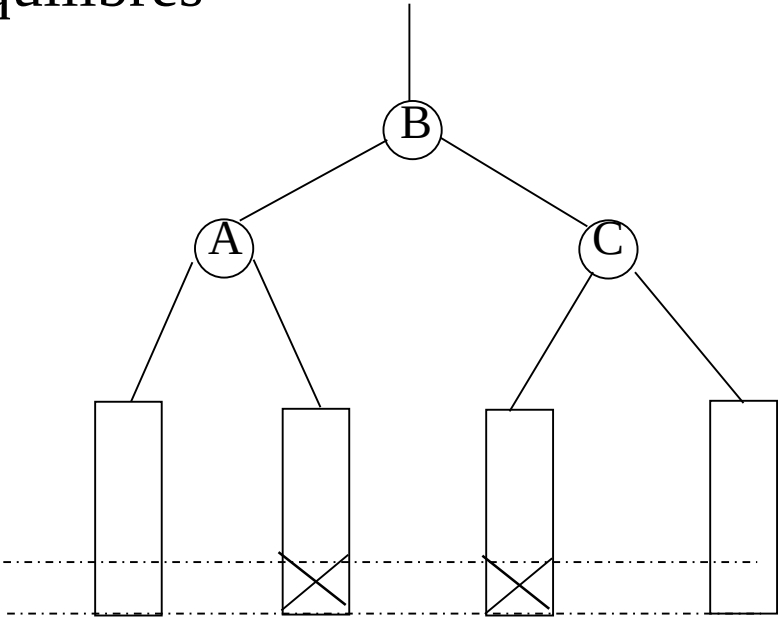
Les Structures de Données

Les arbres

Arbres parfaitement équilibrés



Cas 1



Cas 2

Le résultat de l'équilibrage

Les Structures de Données

Les arbres

Arbres parfaitement équilibrés

```
void Recherche_insertion_equilibrage(int val, ARBRE *Arb, int *bool)
{
    ARBRE *Arb1, *Arb2;

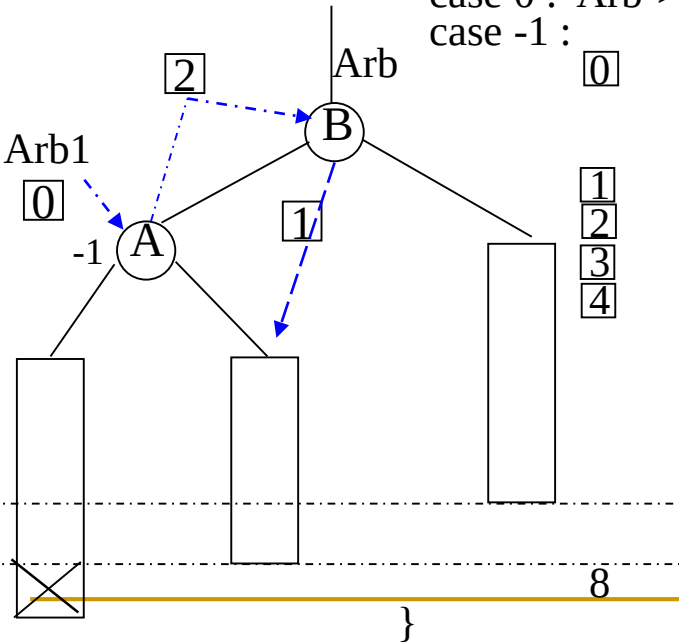
    if (!Arb)                // Arbre vide donc insertion
    {
        Arb = (ARBRE *)malloc(sizeof(ARBRE));
        *bool = 1;            // Le booléen est mis à vrai
        Arb->inf = val;
        Arb->fils_gche = NULL;
        Arb->fils_dt  = NULL;
        Arb->equilibre = 0;
    }
```

Les Structures de Données

Les arbres

Arbres parfaitement équilibrés

```
if (Arb->info > val)
{
    Recherche_insertion_equilibrage(val, Arb->fils_gche, bool);
    if ( bool)
        switch (Arb->equilibre)
        {
            case 1 : Arb->equilibre = 0; *bool = 0;    break;
            case 0 : Arb->equilibre = -1;              break ;
            case -1 :                                  // rééquilibrage
```



```
Arb1 = Arb->fils_gche;
if (Arb1->equilibre == -1)    // Rotation GG simple
{
    Arb->fils_gche = Arb1->fils_dt;
    Arb1->fils_dt = Arb;
    Arb->equilibre = 0;
    Arb = Arb1;
}

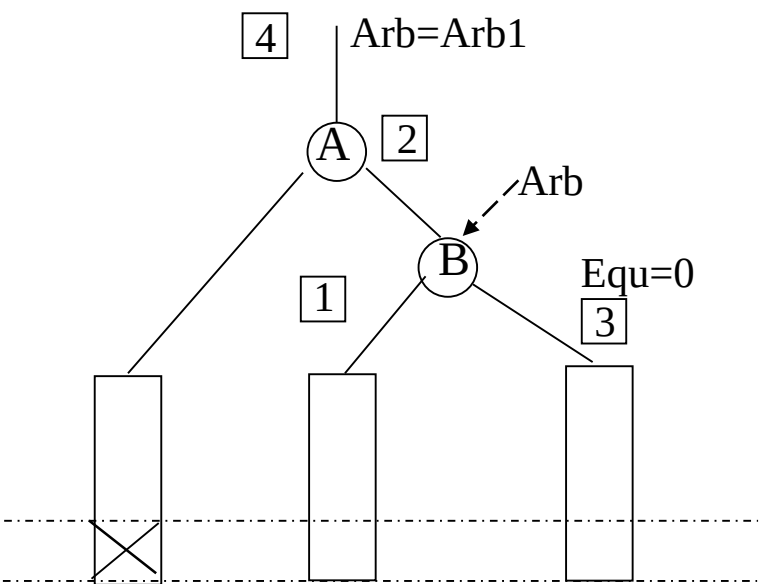
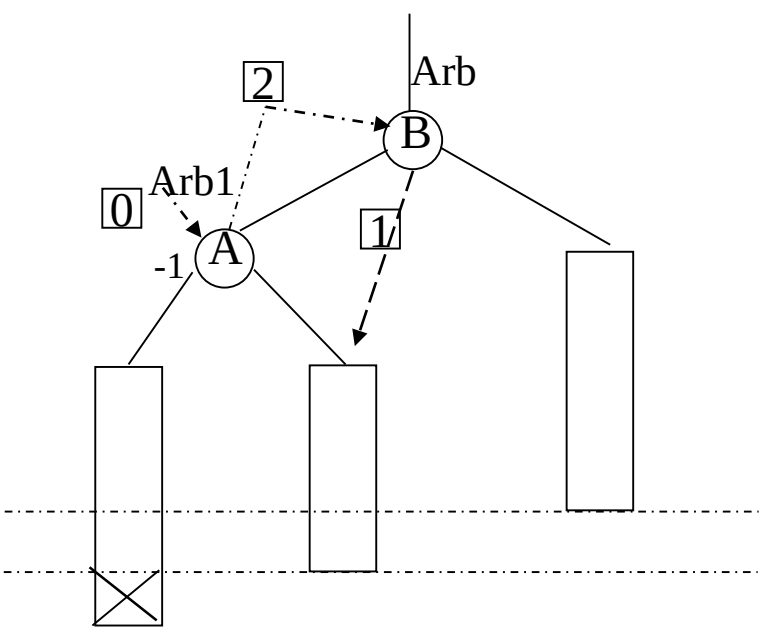
else    // Rotation GD double
{
}

Arb->equilibre = 0; *bool= 0;
```

Les Structures de Données

Les arbres

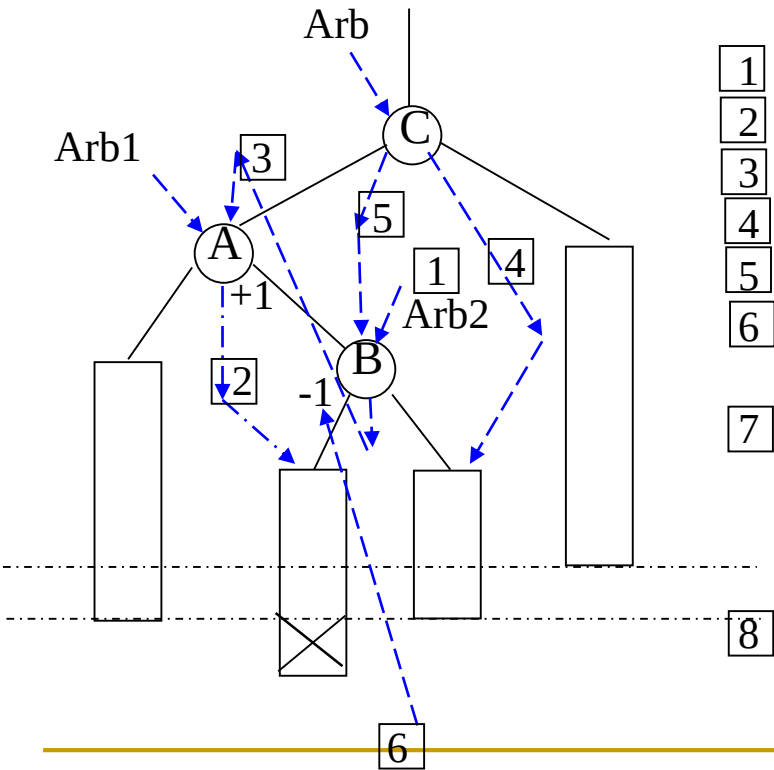
Arbres parfaitement équilibrés



Les Structures de Données

Les arbres

Arbres parfaitement équilibrés

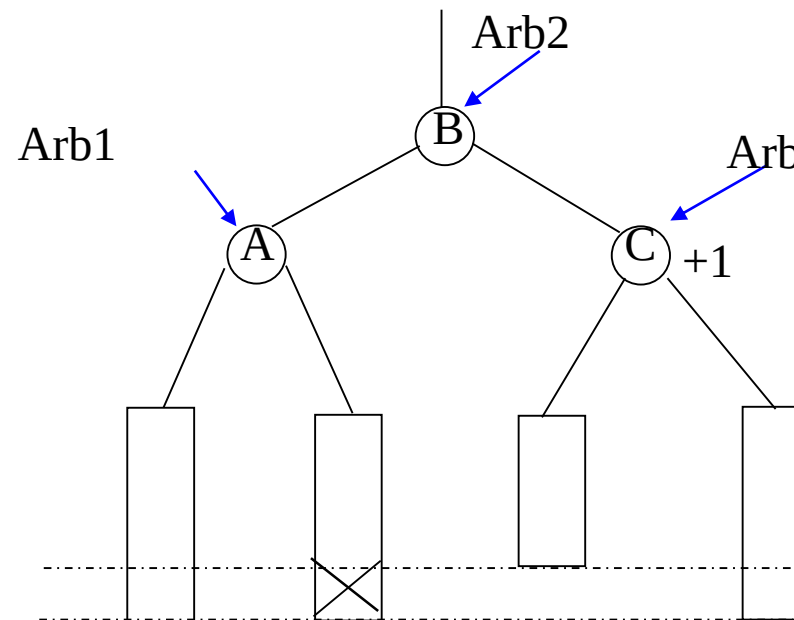


```
else // Rotation GD double
{
    Arb2 = Arb1->fils_dt;
    Arb1->fils_dt = Arb2->fils_gche;
    Arb2->fils_gche = Arb1;
    Arb->fils_gche = Arb2->fils_dt;
    Arb2->fils_dt = Arb;
    if (Arb2->equilbre == -1) Arb->equilibre = 1;
    else Arb->equilibre = 0;
    if (Arb2->equilbre == +1) Arb1->equilibre = -1;
    else Arb1->equilibre = 0;
    Arb = Arb2
}
Arb->equilibre = 0; *bool= 0;
```

Les Structures de Données

Les arbres

Arbres parfaitement équilibrés

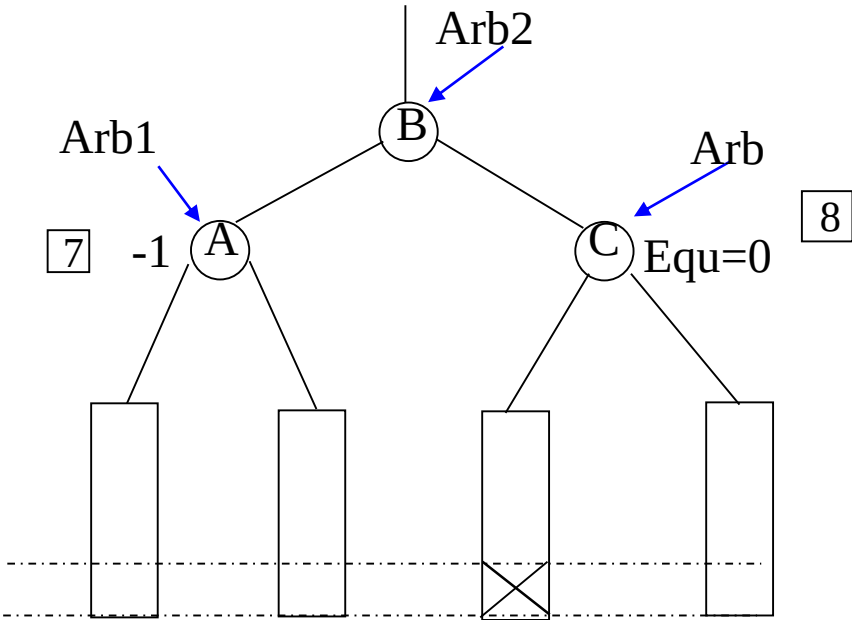
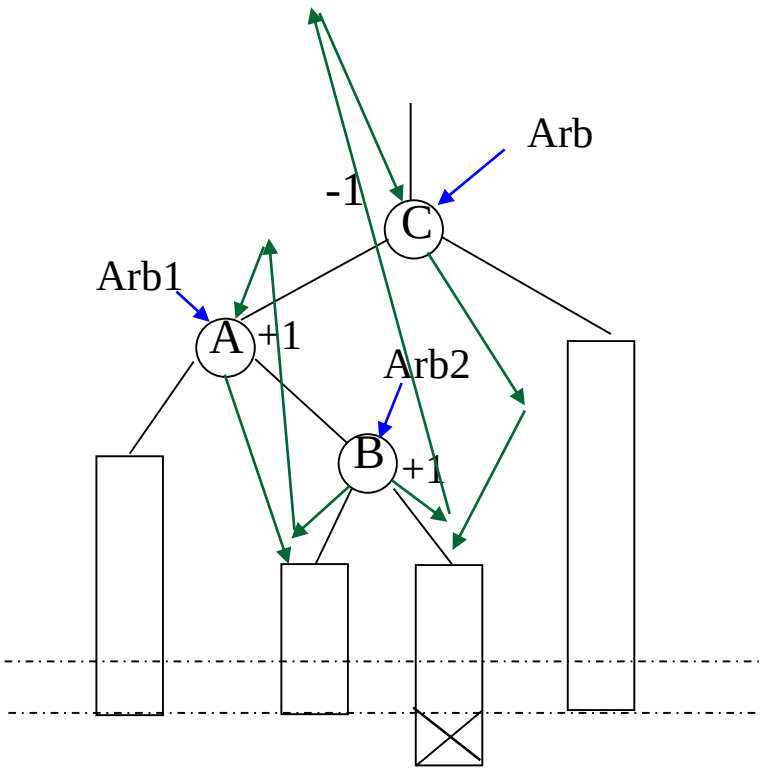


Résultat de l'équilibrage

Les Structures de Données

Les arbres

Arbres parfaitement équilibrés



Résultat de l'équilibrage

Les Structures de Données

Les Structures de Données

Les Structures de Données